

Path Finding for Java

This notebook demonstrates different ways on how path finding algorithms can be utilized for code analysis.

Path algorithms in Graphs are famous for e.g. finding the fastest way from one place to another. How can these be applied to static code analysis and how can the results be interpreted?

One promising algorithm is [All Pairs Shortest Path](#). It shows dependencies from a different perspective and provides an overview on how directly or indirectly dependencies are connected to each other. The longest shortest path has an additional meaning: It is also known as the [Graph Diameter](#) and is very useful as a metric for the complexity of the Graph (or Subgraphs). The longest path (for directed acyclic graphs) can uncover the longest existing (worst case) dependency chains as long as there are no cycles in the Graph.

References

- [jqassistant](#)
- [Neo4j Python Driver](#)
- [All Pairs Shortest Path](#)
- [Longest Path for DAG \(neo4j\)](#)
- [Graph Diameter](#)

What GPT-4 has to say about it

All pairs shortest path

Interpreting the results of the "all pairs shortest path" algorithm on a graph of statically analyzed code modules and their dependencies involves understanding the structure and implications of the paths between nodes (modules) in the graph. Here are some specific steps and insights to consider:

1. **Graph Structure:** Each node represents a code module, and edges indicate dependencies. A directed edge from module A to module B implies that A depends on B.
2. **Shortest Paths:** The results will give you the shortest path lengths between all pairs of modules. This helps identify:

- **Direct Dependencies:** A length of 1 indicates a direct dependency.
 - **Transitive Dependencies:** A length greater than 1 shows indirect dependencies. For example, if the path from A to C is 2, it could mean $A \rightarrow B \rightarrow C$, indicating A indirectly depends on C via B.
3. **Module Isolation:** If a module has very long paths to others, it might be more isolated. This could signal potential issues in the code structure, suggesting that module might be overly complex or decoupled from the rest of the system.
 4. **Critical Paths:** Identify the shortest paths that connect key modules (e.g., core functionalities). These paths can highlight the most crucial dependencies that, if modified, might have extensive impacts on the system.
 5. **Cycle Detection:** If any pairs have paths that loop back to themselves with a length not equal to 0, it indicates a cycle. Cycles can complicate dependency management, potentially leading to recursive dependencies, which can be problematic in terms of maintainability.
 6. **Refactoring Opportunities:** By examining the lengths of paths, you might identify modules that could benefit from refactoring to decrease dependency complexity. For example, a module that has dependencies on many others (with longer path lengths) might be a candidate for breaking into smaller, more manageable components.
 7. **Performance Considerations:** In large systems, long paths could impact performance. If certain modules are far from frequently accessed modules, consider whether they can be optimized for speed.
 8. **Visual Representation:** Creating a visual representation of the graph with the shortest paths highlighted can be immensely helpful. Tools like Graphviz or D3.js can illustrate these relationships clearly, aiding in your analysis.

By focusing on these aspects, you can glean actionable insights from the results of the all pairs shortest path algorithm in the context of your statically analyzed code modules and their dependencies.

Graph diameter (shortest longest path)

The longest shortest path in a dependency graph (often referred to in graph theory as the "diameter" of the graph) represents the maximum distance (in terms of the number of edges or dependencies) between any two nodes (modules) in the graph. Here's how you can interpret this metric in the context of statically analyzed code modules and their dependencies:

1. **Network Complexity:** The longest shortest path indicates the overall complexity of the network of dependencies. A longer path suggests a more complicated interrelationship among modules. For example, a longest shortest path of 6 could

indicate that there is at least one pair of modules in your system that rely on a chain of 6 other modules to communicate or function together.

2. **Potential Bottlenecks:** If the longest shortest path is significant, it may suggest potential bottlenecks in your architecture. For instance, if a core module at the beginning of a long path is slow or error-prone, it could affect numerous other modules dependent on it, resulting in systemic performance issues.
3. **Critical Communication Points:** The endpoints of the longest shortest path can be seen as critical communication points within your codebase. Understanding these connections can help identify which modules should be prioritized for testing and monitoring, especially during changes.
4. **Isolation and Coupling:** A long longest shortest path might indicate that some modules are isolated and far removed from others, which can suggest low cohesion. This can be a sign that the architecture might benefit from refactoring to reduce unnecessary dependencies or to improve modularity.
5. **Refactoring Opportunities:** If the longest shortest path is disproportionately long, it may highlight areas in the codebase where modules are too tightly coupled. This situation presents an opportunity for refactoring to create more independent modules or components that can interact with fewer dependencies.
6. **Impact of Changes:** Modules that lie along or are endpoints of the longest shortest paths are likely to have a significant impact on the overall system. Changes to them should be approached with caution and accompanied by rigorous testing.
7. **Cycle Detection:** In some cases, a long shortest path can indicate the presence of cycles in the graph. If there are paths that seem to loop back on themselves, it suggests potential design flaws that could lead to recursion or infinite loops, complicating maintenance.
8. **Architectural Decisions:** The longest shortest path can inform architectural decisions by providing insights into which dependencies might need to be revised or eliminated. For instance, if certain modules are consistently part of the longest path, it could justify investing resources in redesigning their interactions.

In summary, interpreting the longest shortest path provides a comprehensive view of the interdependencies among modules in your system, focusing on complexity, potential bottlenecks, and opportunities for improvement in architecture and design.

Longest path

1. **Complex Dependencies:** Longest paths indicate modules that have extensive dependencies before reaching another module. For instance, if you find a path like $A \rightarrow B \rightarrow C \rightarrow D$ with a length of 4, it highlights a complex chain of dependencies. This can

suggest that changes in module A might have far-reaching implications across the system.

2. **Potential Bottlenecks:** Modules located at the beginning of long paths could be performance bottlenecks. If a frequently used module is several layers deep in dependencies (e.g., $A \rightarrow B \rightarrow C$), it may slow down the entire system. Optimizing or refactoring these modules could improve performance.
3. **Maintenance Challenges:** Long paths may indicate parts of the code that are difficult to maintain or understand. For example, if module A requires multiple intermediary modules (B, C, D) for its functionality, developers may struggle to trace how changes propagate, leading to potential bugs.
4. **Risk of Change:** A module with a long dependency path can be more risky to modify. If A has dependencies on several modules down the line (e.g., $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$), any changes to A could inadvertently affect E, which might be critical or sensitive. This insight can help prioritize testing and reviews around such modules.
5. **Decoupling Opportunities:** Identifying the longest paths can highlight areas where you might want to break up dependencies. If there's a long chain that could be simplified (e.g., by creating intermediary modules or interfaces), it may lead to a more modular and maintainable architecture.
6. **Redundancy:** Long paths may also reveal redundancy in dependencies. For instance, if multiple modules depend on a series of others ($A \rightarrow B \rightarrow C \rightarrow D$ and $A \rightarrow B \rightarrow E$), it could indicate unnecessary coupling that can be streamlined.
7. **System Understanding:** Long paths can help map out the architecture of your codebase. Understanding which modules are pivotal in long chains can provide insights into the overall design and help inform architectural decisions.
8. **Documentation & Knowledge Transfer:** If certain modules consistently appear in the longest paths, they may require better documentation. Ensuring that their roles and the reasons for their lengthy dependencies are well understood can facilitate knowledge transfer among team members.

By focusing on the longest paths in your dependency graph, you can uncover areas requiring attention for optimization, maintenance, and improved system architecture.

1. Java Packages

1.1 All pairs shortest path

Use "[All Pairs Shortest Path](#)" algorithm to get the shortest path distance between all pairs of dependent Java packages. It shows how many Packages have a direct dependency (distance 1), how many are reachable with one dependency in between (distance 2), and so on...

1.1.1 Create a projection of all Java package dependencies

Creates a in-memory projection of "Java:Package" nodes and their "DEPENDS_ON" relationships as a preparation to run the Graph algorithms. The weight property is not used for now (September 2024) but may be needed for other algorithms/variants some time.

Projected Graph statistics for Java package dependencies

Projection data available: True

nodeCount	relationshipCount	density	sizeInBytes	degreeDistribution.min	degreeDistribution.mean	degreeDistribution.max	degreeDistri
0	116	760	0.056972	2958113	0	6.551724	58

1.1.2 All pairs shortest path in total

First, we'll have a look at the overall/total result of the all pairs shortest path algorithm for all dependencies.

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownRelationshipTypeWarning} {category: UNRECOGNIZED} {title: The provided relationship type is not in the database.} {description: One of the relationship types in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing relationship type is: CONTAINS_PROJECT)} {position: line: 31, column: 39, offset: 1603} for query: "// Path Finding - All pairs shortest path algorithm - Stream - Per project\n\n CALL gds.allShortestPaths.stream(\$dependencies_projection + '-cleaned')\n YIELD sourceNodeId, targetNodeId, distance\n // Filter out all pairs that have no connection (infinite distance)\n WHERE gds.util.isFinite(distance) = true\n WITH toInteger(distance) AS distance\n ,sourceNodeId\n ,targetNodeId\n WHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n // Group by distance to get the overall distribution\n WITH distance\n ,count(*)\n AS distanceTotalPairCount\n ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n // Unwind group to get every corresponding distance, source and target again\n UNWIND sourcesAndTargets AS sourceAndTarget\n WITH *\n ,sourceAndTarget.sourceNodeId\n AS sourceNodeId\n ,sourceAndTarget.targetNodeId AS targetNodeId\n // Resolve node ids to actual nodes\n WITH *\n ,gds.util.asNode(sourceNodeId) AS source\n ,gds.util.asNode(targetNodeId) AS target\n // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n // Optionally get the name of the scan that contained that project\n OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n // Group by project name, if the target project is the same and the distance. Return those as result.\n RETURN sourceProject.name AS sourceProject\n ,sourceScan.name AS sourceScan\n ,source.rootProjectName AS sourceRootProject\n ,(targetProject <> sourceProject) AS isDifferentTargetProject\n ,(targetScan <> sourceScan) AS isDifferentTargetScan\n ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n ,distance\n ,distanceTotalPairCount\n ,distanceTotalSourceCount\n ,distanceTotalTargetCount\n ,count(*) AS pairCount\n ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n ,count(DISTINCT targetNodeId) AS targetNodeCount\n ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n // Sort by source project name, if the target project is the same and the distance, all ascending\n ORDER BY sourceProject, isDifferentTargetProject, distance"

```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: rootProjectName)} {position: line: 38, column: 42, offset: 2086} for query: "// Path Finding - All pairs shortest path algorithm - Stream - Per project\n\n  CALL gds.allShortestPaths.stream($dependencies_projection + '-cleaned')\n  YIELD sourceNodeId, targetNodeId, distance\n  // Filter out all pairs that have no connection (infinite distance)\n  WHERE gds.util.isFinite(distance) = true\n  WITH toInteger(distance) AS distance\n  ,sourceNodeId\n  ,targetNodeId\n  WHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n  // Group by distance to get the overall distribution\n  WITH distance\n  ,count(*) AS distanceTotalPairCount\n  ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n  ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n  ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n  // Unwind group to get every corresponding distance, source and target again\n  UNWIND sourcesAndTargets AS sourceAndTarget\n  WITH *\n  ,sourceAndTarget.sourceNodeId AS sourceNodeId\n  ,sourceAndTarget.targetNodeId AS targetNodeId\n  // Resolve node ids to actual nodes\n  WITH *\n  ,gds.util.asNode(sourceNodeId) AS source\n  ,gds.util.asNode(targetNodeId) AS target\n  // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n  OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n  OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n  // Optionally get the name of the scan that contained that project\n  OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n  OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n  // Group by project name, if the target project is the same and the distance. Return those as result.\n  RETURN sourceProject.name AS sourceProject\n  ,sourceScan.name AS sourceScan\n  ,source.rootProjectName AS sourceRootProject\n  ,(targetProject <> sourceProject) AS isDifferentTargetProject\n  ,(targetScan <> sourceScan) AS isDifferentTargetScan\n  ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n  ,distance\n  ,distanceTotalPairCount\n  ,distanceTotalSourceCount\n  ,distanceTotalTargetCount\n  ,count(*) AS pairCount\n  ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n  ,count(DISTINCT targetNodeId) AS targetNodeCount\n  ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n  ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n  ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n  // Sort by source project name, if the target project is the same and the distance, all ascending\n  ORDER BY sourceProject, isDifferentTargetProject, distance"

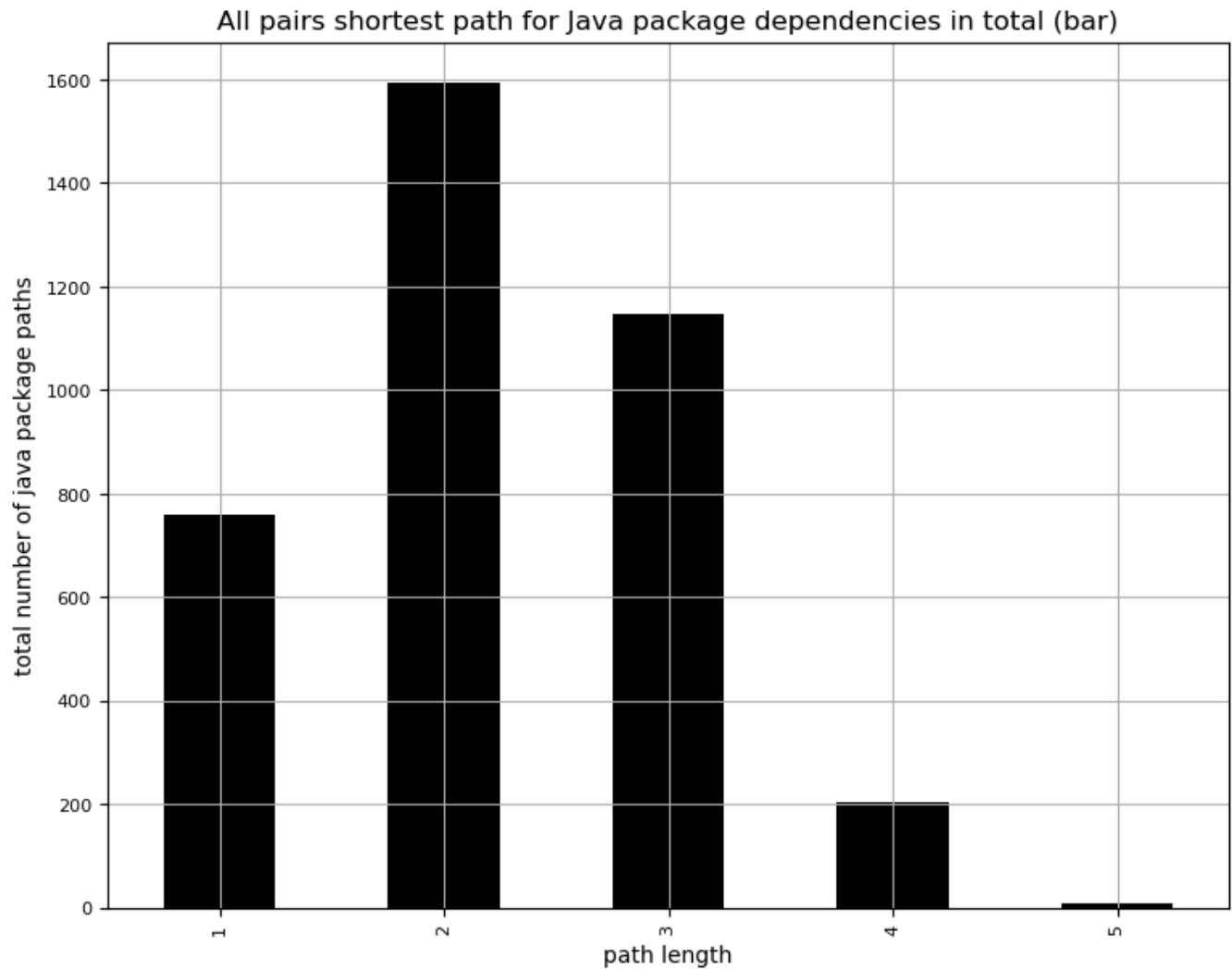
```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: rootProjectName)} {position: line: 38, column: 16, offset: 2060} for query: "// Path Finding - All pairs shortest path algorithm - Stream - Per project\n\n CALL gds.allShortestPaths.stream(\$dependencies_projection + '-cleaned')\n YIELD sourceNodeId, targetNodeId, distance\n // Filter out all pairs that have no connection (infinite distance)\n WHERE gds.util.isFinite(distance) = true\n WITH toInteger(distance) AS distance\n ,sourceNodeId\n ,targetNodeId\n WHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n // Group by distance to get the overall distribution\n WITH distance\n ,count(*) AS distanceTotalPairCount\n ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n // Unwind group to get every corresponding distance, source and target again\n UNWIND sourcesAndTargets AS sourceAndTarget\n WITH *\n ,sourceAndTarget.sourceNodeId AS sourceNodeId\n ,sourceAndTarget.targetNodeId AS targetNodeId\n // Resolve node ids to actual nodes\n WITH *\n ,gds.util.asNode(sourceNodeId) AS source\n ,gds.util.asNode(targetNodeId) AS target\n // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n // Optionally get the name of the scan that contained that project\n OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n // Group by project name, if the target project is the same and the distance. Return those as result.\n RETURN sourceProject.name AS sourceProject\n ,sourceScan.name AS sourceScan\n ,source.rootProjectName AS sourceRootProject\n ,(targetProject <> sourceProject) AS isDifferentTargetProject\n ,(targetScan <> sourceScan) AS isDifferentTargetScan\n ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n ,distance\n ,distanceTotalPairCount\n ,distanceTotalSourceCount\n ,distanceTotalTargetCount\n ,count(*) AS pairCount\n ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n ,count(DISTINCT targetNodeId) AS targetNodeCount\n ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n // Sort by source project name, if the target project is the same and the distance, all ascending\n ORDER BY sourceProject, isDifferentTargetProject, distance"

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: rootProjectName)} {position: line: 35, column: 15, offset: 1865} for query: "// Path Finding - All pairs shortest path algorithm - Stream - Per project\n\n CALL gds.allShortestPaths.stream(\$dependencies_projection + '-cleaned')\n YIELD sourceNodeId, targetNodeId, distance\n // Filter out all pairs that have no connection (infinite distance)\n WHERE gds.util.isFinite(distance) = true\n WITH toInteger(distance) AS distance\n ,sourceNodeId\n ,targetNodeId\n WHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n // Group by distance to get the overall distribution\n WITH distance\n ,count(*) AS distanceTotalPairCount\n ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n // Unwind group to get every corresponding distance, source and target again\n UNWIND sourcesAndTargets AS sourceAndTarget\n WITH *\n ,sourceAndTarget.sourceNodeId AS sourceNodeId\n ,sourceAndTarget.targetNodeId AS targetNodeId\n // Resolve node ids to actual nodes\n WITH *\n ,gds.util.asNode(sourceNodeId) AS source\n ,gds.util.asNode(targetNodeId) AS target\n // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n // Optionally get the name of the scan that contained that project\n OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n // Group by project name, if the target project is the same and the distance. Return those as result.\n RETURN sourceProject.name AS sourceProject\n ,sourceScan.name AS sourceScan\n ,source.rootProjectName AS sourceRootProject\n ,(targetProject <> sourceProject) AS isDifferentTargetProject\n ,(targetScan <> sourceScan) AS isDifferentTargetScan\n ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n ,distance\n ,distanceTotalPairCount\n ,distanceTotalSourceCount\n ,distanceTotalTargetCount\n ,count(*) AS pairCount\n ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n ,count(DISTINCT targetNodeId) AS targetNodeCount\n ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n // Sort by source project name, if the target project is the same and the distance, all ascending\n ORDER BY sourceProject, isDifferentTargetProject, distance"

	index	distance	distanceTotalPairCount	distanceTotalSourceCount	distanceTotalTargetCount
0	0	1	760	110	103
1	1	2	1593	101	88
2	4	3	1146	97	60
3	5	4	204	61	21
4	18	5	7	3	5

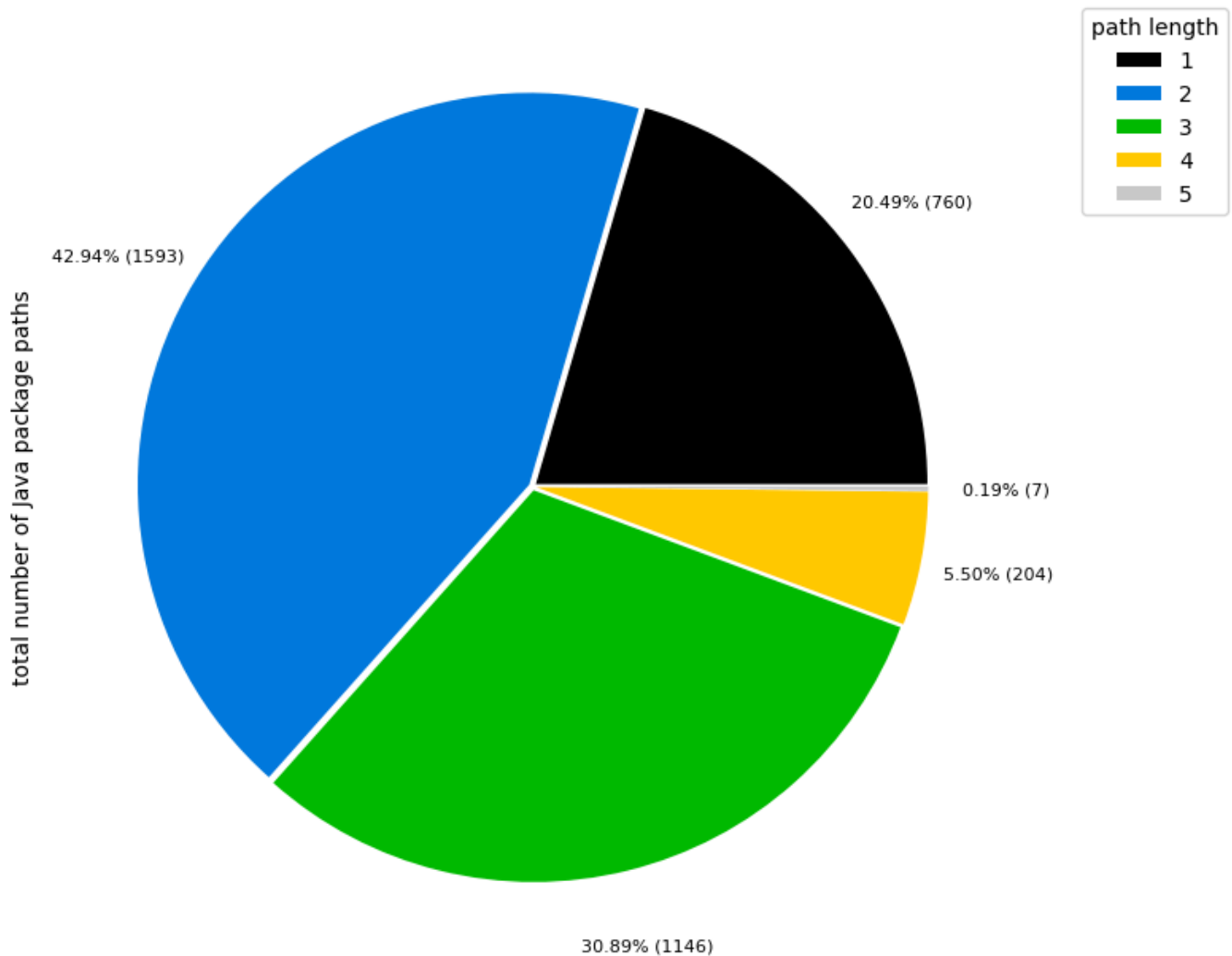
All pairs shortest path in total - Path count per length - Bar chart



All pairs shortest path in total - Path count per length - Pie chart

<Figure size 640x480 with 0 Axes>

All pairs shortest path for Java package dependencies in total (pie)



1.1.3 All pairs shortest path in detail

The following table shows the first 10 rows with all details of the query above. It contains the results of the "all pairs shortest path" algorithm including the artifact the source node belong to and if the target node is the same or not. The main intuition here is to show how the data is structured. It provides the basis for tables and charts shown in following sections below, that filter and group the data accordingly.

	sourceProject	sourceScan	sourceRootProject	isDifferentTargetProject	isDifferentTargetScan	isDifferentTargetRootProject	distance	distance
0	axon-configuration-4.11.1	None	None	True	None	None	1	
1	axon-configuration-4.11.1	None	None	True	None	None	2	
2	axon-disruptor-4.11.1	None	None	True	None	None	1	
3	axon-disruptor-4.11.1	None	None	True	None	None	2	
4	axon-disruptor-4.11.1	None	None	True	None	None	3	
5	axon-disruptor-4.11.1	None	None	True	None	None	4	
6	axon-eventsourcing-4.11.1	None	None	False	None	None	1	
7	axon-eventsourcing-4.11.1	None	None	False	None	None	2	
8	axon-eventsourcing-4.11.1	None	None	False	None	None	3	
9	axon-eventsourcing-4.11.1	None	None	False	None	None	4	

1.1.4 All pairs shortest path for each artifact

In this section we'll focus only on pairs of nodes that both belong to the same artifact, filtering out every line that has `isDifferentTargetProject==False`. The first ten rows

are shown in a table followed by charts that show the distribution of shortest path distances across different artifacts in stacked bar charts (absolute and normalized).

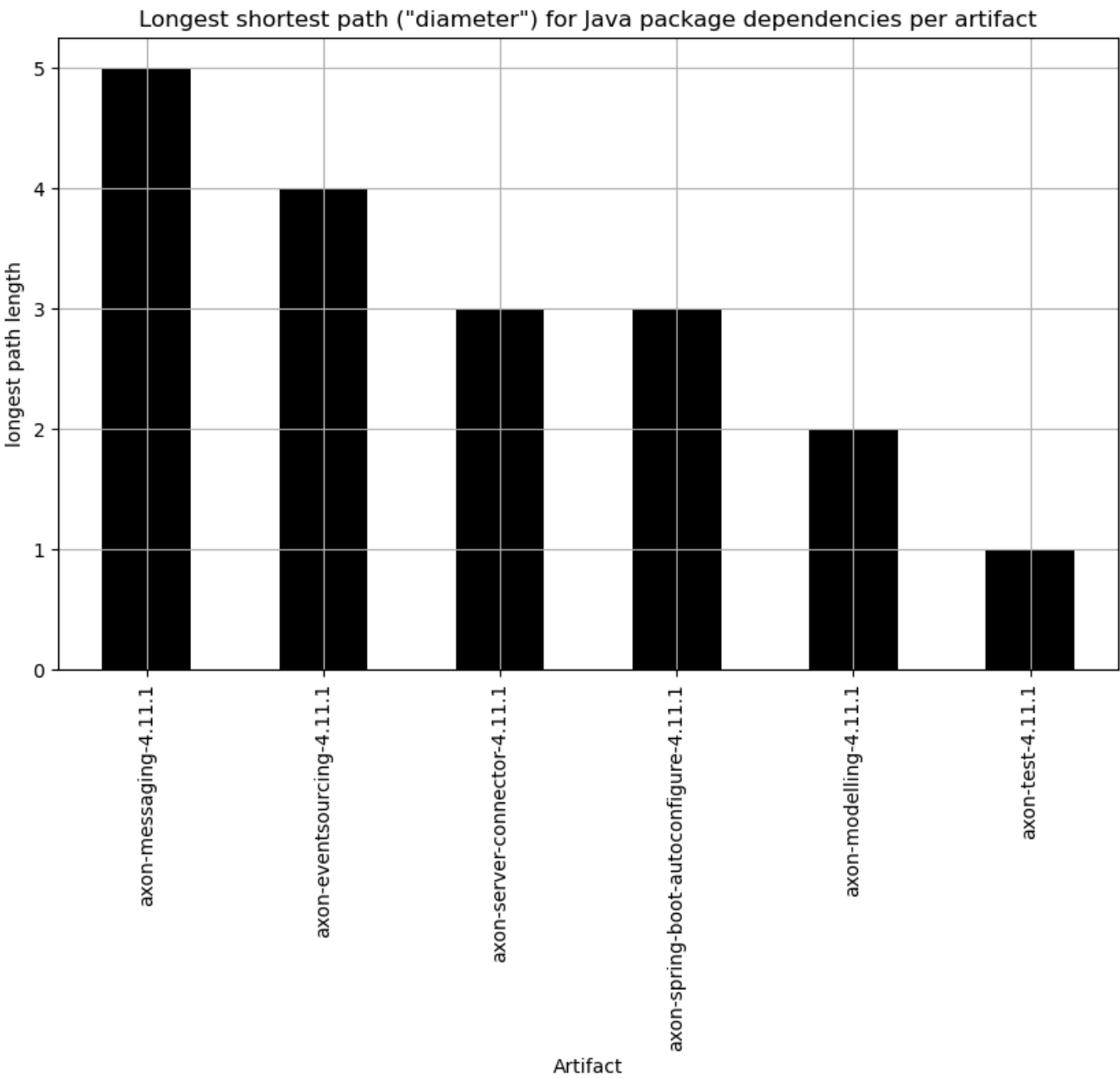
Note: It is possible that a (shortest) path could have nodes in between that belong to different artifacts. Therefore, the data of each artifact isn't perfectly isolated. However, it shows how the dependencies interact across artifacts "in real life" while still providing a decent isolation of each artifact.

	sourceProject	sourceScan	sourceRootProject	isDifferentTargetProject	isDifferentTargetScan	isDifferentTargetRootProject	distance	distal
6	axon-eventsourcing-4.11.1	None	None	False	None	None	1	
7	axon-eventsourcing-4.11.1	None	None	False	None	None	2	
8	axon-eventsourcing-4.11.1	None	None	False	None	None	3	
9	axon-eventsourcing-4.11.1	None	None	False	None	None	4	
14	axon-messaging-4.11.1	None	None	False	None	None	1	
15	axon-messaging-4.11.1	None	None	False	None	None	2	
16	axon-messaging-4.11.1	None	None	False	None	None	3	
17	axon-messaging-4.11.1	None	None	False	None	None	4	
18	axon-messaging-4.11.1	None	None	False	None	None	5	
19	axon-modelling-4.11.1	None	None	False	None	None	1	

All pairs shortest path for each artifact - Longest shortest path (Diameter) for each artifact

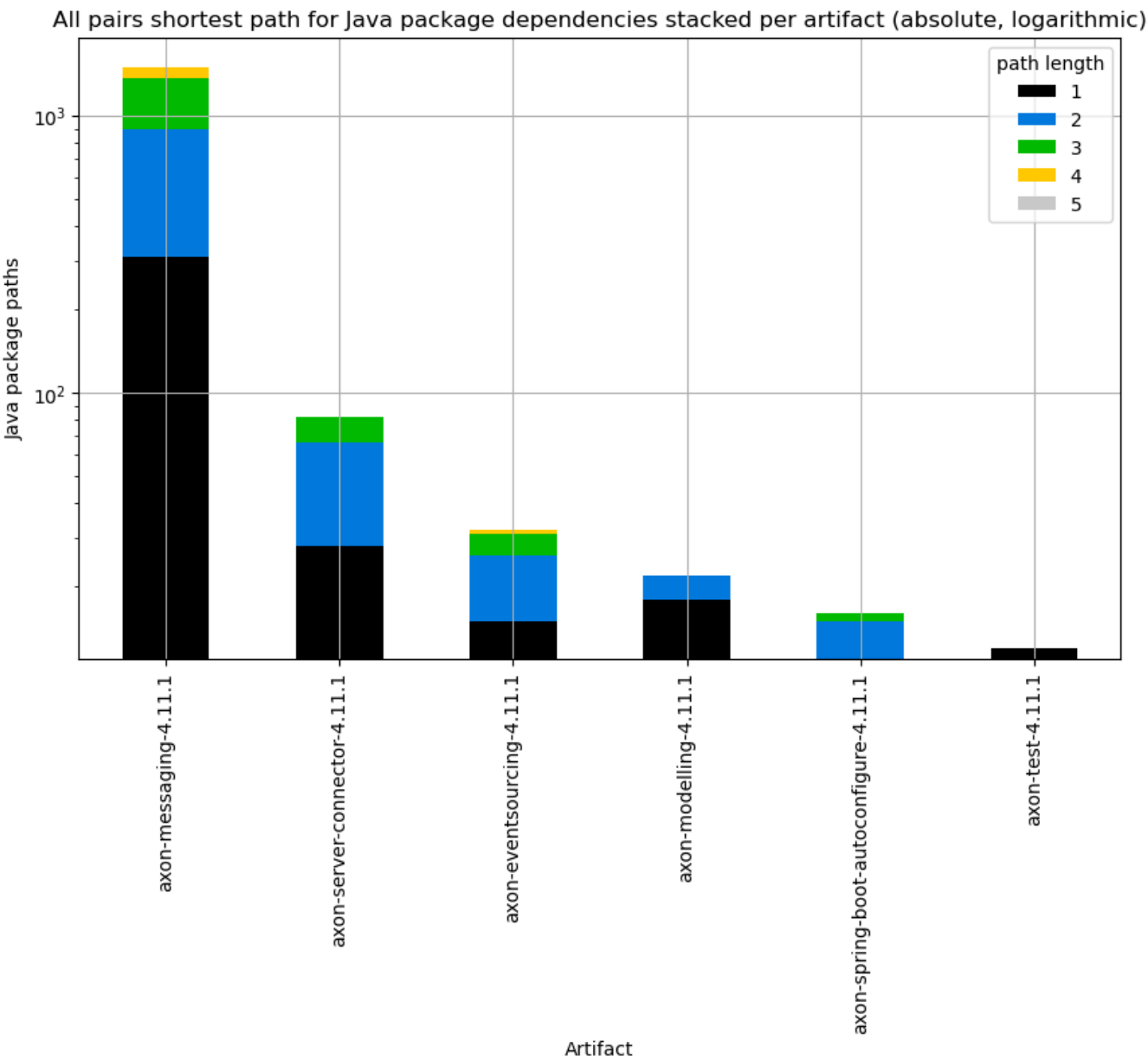
Shows the top 20 artifacts with the longest shortest path (=Graph Diameter).

sourceProject	
axon-messaging-4.11.1	5
axon-eventsourcing-4.11.1	4
axon-server-connector-4.11.1	3
axon-spring-boot-autoconfigure-4.11.1	3
axon-modelling-4.11.1	2
axon-test-4.11.1	1
Name: distance, dtype: int64	



All pairs shortest path for each artifact - Bar chart (absolute)

<Figure size 640x480 with 0 Axes>

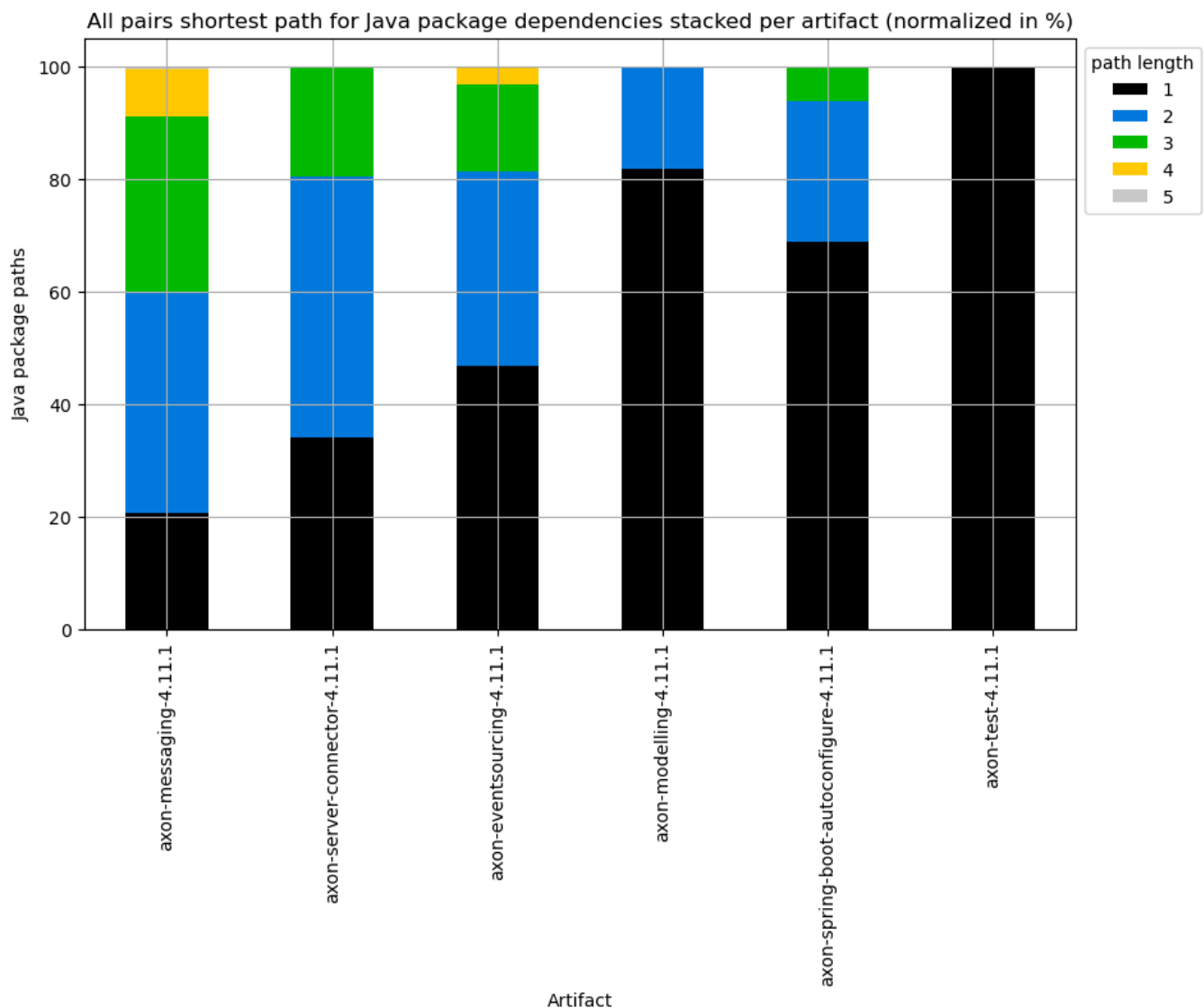


All pairs shortest path for each artifact - Bar chart (normalized)

Shows the top 50 artifacts with the highest number of dependency paths stacked by their length.

distance	1	2	3	4	5
sourceProject					
axon-messaging-4.11.1	20.719520	39.040640	31.245836	8.527648	0.466356
axon-server-connector-4.11.1	34.146341	46.341463	19.512195	0.000000	0.000000
axon-eventsourcing-4.11.1	46.875000	34.375000	15.625000	3.125000	0.000000
axon-modelling-4.11.1	81.818182	18.181818	0.000000	0.000000	0.000000
axon-spring-boot-autoconfigure-4.11.1	68.750000	25.000000	6.250000	0.000000	0.000000
axon-test-4.11.1	100.000000	0.000000	0.000000	0.000000	0.000000

<Figure size 640x480 with 0 Axes>



1.2 Longest path

Use [Longest Path](#) algorithm to get the longest paths between Java packages. It is typically higher than the longest shortest path (diameter) and helps together with it to get a good overview of the complexity.

Note: This algorithm requires a Directed Acyclic Graph (DAG) and will lead to inaccurate results when the Graph contains cycles.

1.2.1 Longest path in total

First, we'll have a look at the overall/total result of the longest path algorithm for all dependencies.

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: rootProjectName)} {position: line: 36, column: 42, offset: 2007} for query: "// Longest paths distribution\n\n  CALL gds.dag.longestPath.stream($dependencies_projection + '-cleaned')\n  YIELD index, sourceNode, targetNode, totalCost//, nodeIds, costs, path\n  WITH toInteger(totalCost) AS distance\n  AS sourceNodeId\n  ,targetNode\n  AS targetNodeId\n  WHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n  // Group by distance to get the overall distribution\n  WITH distance\n  ,count(*)\n  AS distanceTotalPairCount\n  ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n  ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n  ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n  // Unwind group to get every corresponding distance, source and target again\n  UNWIND sourcesAndTargets AS sourceAndTarget\n  WITH *\n  ,sourceAndTarget.sourceNodeId AS sourceNodeId\n  ,sourceAndTarget.targetNodeId AS targetNodeId\n  // Resolve node ids to actual nodes\n  WITH *\n  ,gds.util.asNode(sourceNodeId) AS source\n  ,gds.util.asNode(targetNodeId) AS target\n  // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n  OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n  OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n  // Optionally get the name of the scan that contained that project\n  OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n  OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n  // Group by project name, if the target project is the same and the distance. Return those as result.\n  RETURN sourceProject.name\n  AS sourceProject\n  ,sourceScan.name\n  AS sourceScan\n  ,source.rootProjectName\n  AS sourceRootProject\n  ,(targetProject <> sourceProject) AS isDifferentTargetProject\n  ,(targetScan <> sourceScan) AS isDifferentTargetScan\n  ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n  ,distance\n  ,distanceTotalPairCount\n  ,distanceTotalSourceCount\n  ,distanceTotalTargetCount\n  ,count(*)\n  AS pairCount\n  ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n  ,count(DISTINCT targetNodeId) AS targetNodeCount\n  ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n  ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n  ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n  // Sort by source project name, if the target project is the same and the distance, all ascending\n  ORDER BY sourceProject, isDifferentTargetProject, distance\n\n\n\n\n\n\n//RETURN toInteger(totalCost) AS totalCost\n//\n,count(*)\nAS nodeCount\n//ORDER BY totalCost"
```

```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: rootProjectName)} {position: line: 36, column: 16, offset: 1981} for query: "// Longest paths distribution\n\n  CALL gds.dag.longestPath.stream($dependencies_projection + '-cleaned')\n  YIELD index, sourceNode, targetNode, totalCost//,\n  nodeIds, costs, path\n  WITH toInteger(totalCost) AS distance\n  ,sourceNode\n  AS sourceNodeId\n  ,targetNode\n  AS targetNodeId\n  WHERE sourceNodeId\n  <> targetNodeId // Filter out cyclic dependencies\n  // Group by distance to get the overall distribution\n  WITH distance\n  ,count(*)\n  AS distanceTotalPairCount\n  ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n  ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n  ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n  // Unwind group to get every corresponding distance, source and target again\n  UNWIND sourcesAndTargets AS sourceAndTarget\n  WITH *\n  ,sourceAndTarget.sourceNodeId AS sourceNodeId\n  ,sourceAndTarget.targetNodeId AS targetNodeId\n  // Resolve node ids to actual nodes\n  WITH *\n  ,gds.util.asNode(sourceNodeId) AS source\n  ,gds.util.asNode(targetNodeId) AS target\n  // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n  OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n  OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n  // Optionally get the name of the scan that contained that project\n  OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n  OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n  // Group by project name, if the target project is the same and the distance. Return those as result.\n  RETURN sourceProject.name\n  AS sourceProject\n  ,sourceScan.name\n  AS sourceScan\n  ,source.rootProjectName\n  AS sourceRootProject\n  ,(targetProject <> sourceProject) AS isDifferentTargetProject\n  ,(targetScan <> sourceScan)\n  AS isDifferentTargetScan\n  ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n  ,distance\n  ,distanceTotalPairCount\n  ,distanceTotalSourceCount\n  ,distanceTotalTargetCount\n  ,count(*)\n  AS pairCount\n  ,count(DISTINCT sourceNodeId)\n  AS sourceNodeCount\n  ,count(DISTINCT targetNodeId)\n  AS targetNodeCount\n  ,collect(DISTINCT source.fileName\n  + ' -> ' + target.fileName)[0..4]\n  AS examples\n  ,collect(DISTINCT sourceProject.name\n  + ' -> ' + targetProject.name)[0..4]\n  AS exampleProjects\n  ,collect(DISTINCT sourceScan.name\n  + ' -> ' + targetScan.name)[0..4]\n  AS exampleScans\n  // Sort by source project name, if the target project is the same and the distance, all ascending\n  ORDER BY sourceProject, isDifferentTargetProject, distance\n  \n\n\n\n\n\n\n\n\n\n  //RETURN toInteger(totalCost) AS totalCost\n  //\n  ,count(*)\n  AS nodeCount\n  //ORDER BY totalCost"

```

```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownRelationshipTypeWarning} {category: UNRECOGNIZED} {title: The provided relationship type is not in the database.} {description: One of the relationship types in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing relationship type is: CONTAINS_PROJECT)} {position: line: 29, column: 39, offset: 1524} for query: "// Longest paths distribution\n\n    CALL gds.dag.longestPath.stream($dependencies_projection + '-cleaned')\n    YIELD index, sourceNode, targetNode, totalCost//, nodeIds, costs, path\n    WITH toInteger(totalCost) AS distance\n    ,sourceNode AS sourceNodeId\n    ,targetNode AS targetNodeId\nWHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n// Group by distance to get the overall distribution\n    WITH distance\n    ,count(*)\nAS distanceTotalPairCount\n    ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n    ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n    ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n// Unwind group to get every corresponding distance, source and target again\nUNWIND sourcesAndTargets AS sourceAndTarget\n    WITH *\n    ,sourceAndTarget.sourceNodeId AS sourceNodeId\n    ,sourceAndTarget.targetNodeId AS targetNodeId\n// Resolve node ids to actual nodes\n    WITH *\n    ,gds.util.asNode(sourceNodeId) AS source\n    ,gds.util.asNode(targetNodeId) AS target\n// Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\nOPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\nOPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n// Optionally get the name of the scan that contained that project\nOPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\nOPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n// Group by project name, if the target project is the same and the distance. Return those as result.\nRETURN sourceProject.name AS sourceProject\n    ,sourceScan.name AS sourceScan\n    ,source.rootProjectName AS sourceRootProject\n    ,(targetProject <> sourceProject) AS isDifferentTargetProject\n    ,(targetScan <> sourceScan) AS isDifferentTargetScan\n    ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n    ,distance\n    ,distanceTotalPairCount\n    ,distanceTotalSourceCount\n    ,distanceTotalTargetCount\n    ,count(*) AS pairCount\n    ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n    ,count(DISTINCT targetNodeId) AS targetNodeCount\n    ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n    ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n    ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n// Sort by source project name, if the target project is the same and the distance, all ascending\nORDER BY sourceProject, isDifferentTargetProject, distance\n\n\n\n\n\n\n//RETURN toInteger(totalCost) AS totalCost\n//\n    ,count(*) AS nodeCount\n//ORDER BY totalCost"

```

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: rootProjectName)} {position: line: 33, column: 15, offset: 1786} for query: "// Longest paths distribution\n\n    CALL gds.dag.longestPath.stream($dependencies_projection + '-cleaned')\n    YIELD index, sourceNode, targetNode, totalCost//, nodeIds, costs, path\n    WITH toInteger(totalCost) AS distance\n    AS sourceNodeId\n    ,targetNode\n    AS targetNodeId\n    WHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n    // Group by distance to get the overall distribution\n    WITH distance\n    ,count(*)\n    AS distanceTotalPairCount\n    ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n    ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n    ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n    // Unwind group to get every corresponding distance, source and target again\n    UNWIND sourcesAndTargets AS sourceAndTarget\n    WITH *\n    ,sourceAndTarget.sourceNodeId AS sourceNodeId\n    ,sourceAndTarget.targetNodeId AS targetNodeId\n    // Resolve node ids to actual nodes\n    WITH *\n    ,gds.util.asNode(sourceNodeId) AS source\n    ,gds.util.asNode(targetNodeId) AS target\n    // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n    OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n    OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n    // Optionally get the name of the scan that contained that project\n    OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n    OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n    // Group by project name, if the target project is the same and the distance. Return those as result.\n    RETURN sourceProject.name\n    AS sourceProject\n    ,sourceScan.name\n    AS sourceScan\n    ,source.rootProjectName\n    AS sourceRootProject\n    ,(targetProject <> sourceProject) AS isDifferentTargetProject\n    ,(targetScan <> sourceScan) AS isDifferentTargetScan\n    ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n    ,distance\n    ,distanceTotalPairCount\n    ,distanceTotalSourceCount\n    ,distanceTotalTargetCount\n    ,count(*)\n    AS pairCount\n    ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n    ,count(DISTINCT targetNodeId) AS targetNodeCount\n    ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n    ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n    ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n    // Sort by source project name, if the target project is the same and the distance, all ascending\n    ORDER BY sourceProject, isDifferentTargetProject, distance\n\n\n\n\n\n\n//RETURN toInteger(totalCost) AS totalCost\n//\n,count(*)\nAS nodeCount\n//ORDER BY totalCost"
```



```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownRelationshipTypeWarning} {category: UNRECOGNIZED} {title: The provided relationship type is not in the database.} {description: One of the relationship types in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing relationship type is: CONTAINS_PROJECT)} {position: line: 28, column: 39, offset: 1451} for query: "// Longest paths distribution\n\n    CALL gds.dag.longestPath.stream($dependencies_projection + '-cleaned')\n    YIELD index, sourceNode, targetNode, totalCost//, nodeIds, costs, path\n    WITH toInteger(totalCost) AS distance\n    ,sourceNode AS sourceNodeId\n    ,targetNode AS targetNodeId\nWHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n// Group by distance to get the overall distribution\n    WITH distance\n    ,count(*)\nAS distanceTotalPairCount\n    ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n    ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n    ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n// Unwind group to get every corresponding distance, source and target again\nUNWIND sourcesAndTargets AS sourceAndTarget\n    WITH *\n    ,sourceAndTarget.sourceNodeId AS sourceNodeId\n    ,sourceAndTarget.targetNodeId AS targetNodeId\n// Resolve node ids to actual nodes\n    WITH *\n    ,gds.util.asNode(sourceNodeId) AS source\n    ,gds.util.asNode(targetNodeId) AS target\n// Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\nOPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\nOPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n// Optionally get the name of the scan that contained that project\nOPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\nOPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n// Group by project name, if the target project is the same and the distance. Return those as result.\nRETURN sourceProject.name AS sourceProject\n    ,sourceScan.name AS sourceScan\n    ,source.rootProjectName AS sourceRootProject\n    ,(targetProject <> sourceProject) AS isDifferentTargetProject\n    ,(targetScan <> sourceScan) AS isDifferentTargetScan\n    ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n    ,distance\n    ,distanceTotalPairCount\n    ,distanceTotalSourceCount\n    ,distanceTotalTargetCount\n    ,count(*) AS pairCount\n    ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n    ,count(DISTINCT targetNodeId) AS targetNodeCount\n    ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n    ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n    ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n// Sort by source project name, if the target project is the same and the distance, all ascending\nORDER BY sourceProject, isDifferentTargetProject, distance\n\n\n\n\n\n\n//RETURN toInteger(totalCost) AS totalCost\n//\n    ,count(*) AS nodeCount\n//ORDER BY totalCost"

```

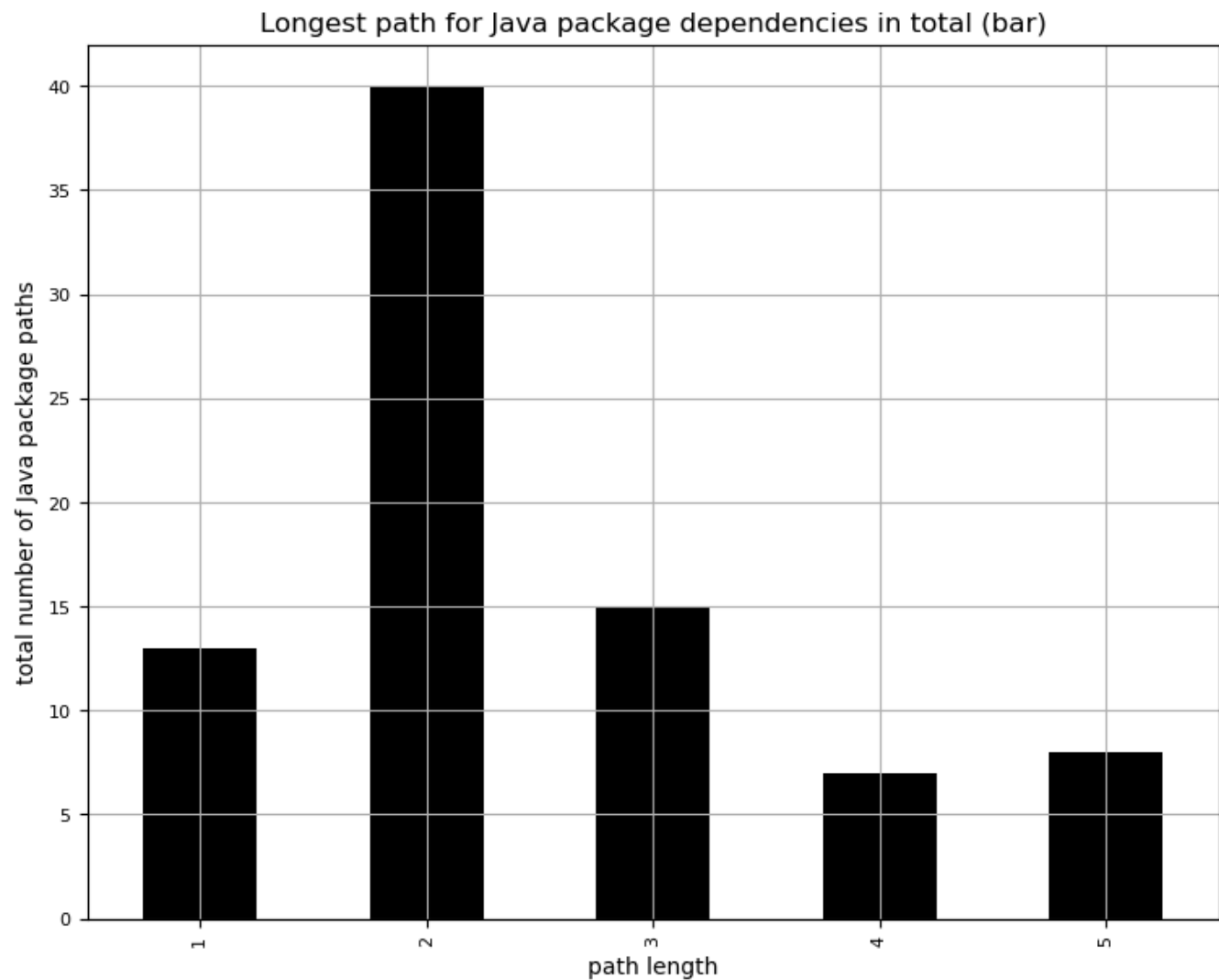
Longest path in total - Max longest path

The max. longest path of the projected package dependencies is: 5

Longest path in total - Paths per length - Table

	index	distance	distanceTotalPairCount	distanceTotalSourceCount	distanceTotalTargetCount
0	0	1	13	5	13
1	2	2	40	3	40
2	5	3	15	2	15
3	9	4	7	1	7
4	10	5	8	1	8

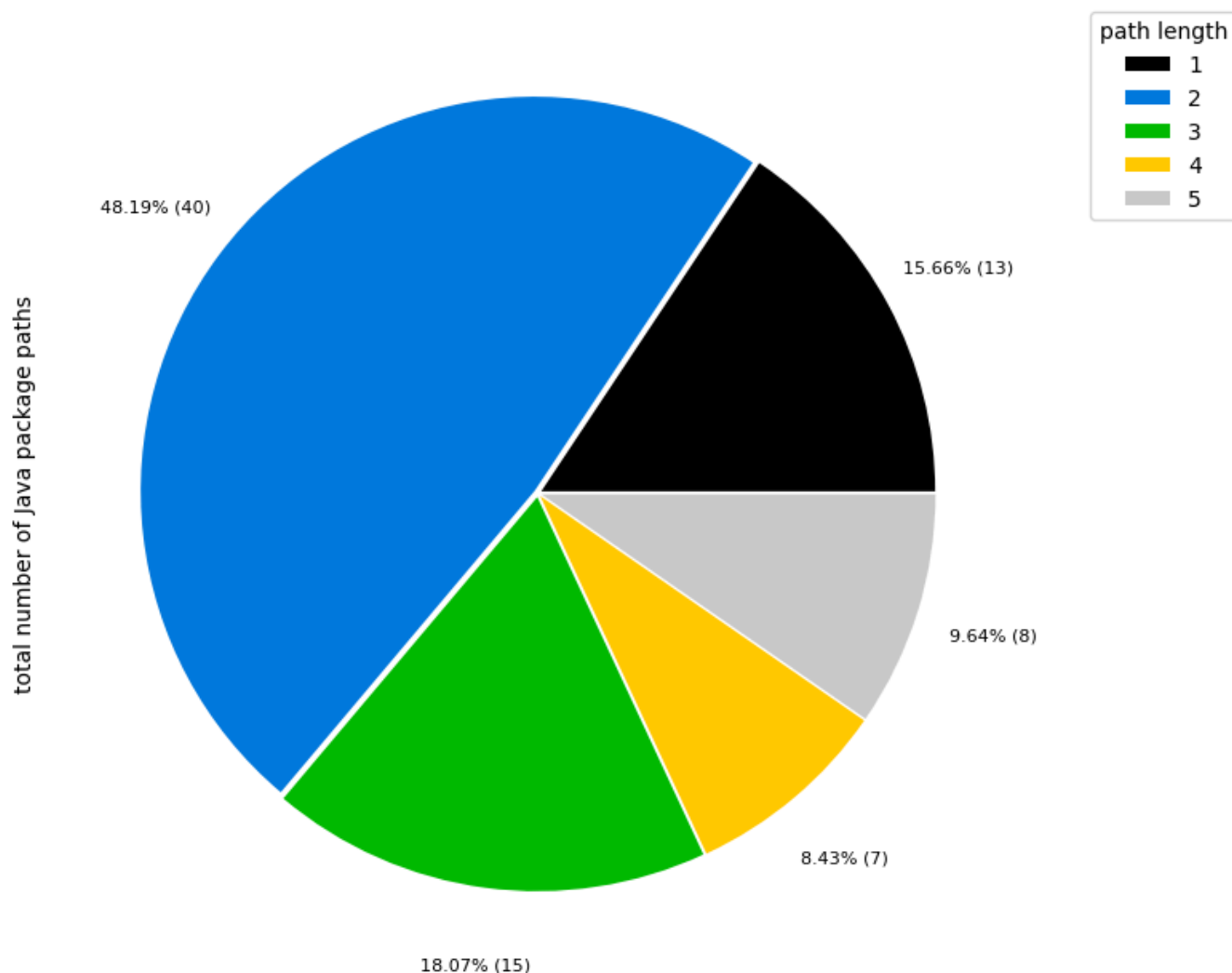
Longest path in total - Path count per length - Bar chart



Longest path in total - Path count per length - Pie chart

<Figure size 640x480 with 0 Axes>

Longest path for Java package dependencies in total (pie)



1.2.2 Longest path in detail

The following table shows the first 10 rows with all details of the query above. It contains the results of the "longest path" algorithm including the artifact the source node belongs to and if the target node is in the same artifact or not. The main intuition is to show how the data is structured. It provides the basis for tables and charts shown in following sections below, that filter and group the data accordingly.

	sourceProject	sourceScan	sourceRootProject	isDifferentTargetProject	isDifferentTargetScan	isDifferentTargetRootProject	distance	distance
0	axon-modelling-4.11.1	None	None	True	None	None	1	
1	axon-server-connector-4.11.1	None	None	False	None	None	1	
2	axon-server-connector-4.11.1	None	None	False	None	None	2	
3	axon-spring-boot-autoconfigure-4.11.1	None	None	False	None	None	1	
4	axon-spring-boot-autoconfigure-4.11.1	None	None	False	None	None	2	
5	axon-spring-boot-autoconfigure-4.11.1	None	None	False	None	None	3	
6	axon-spring-boot-autoconfigure-4.11.1	None	None	True	None	None	1	
7	axon-spring-boot-autoconfigure-4.11.1	None	None	True	None	None	2	
8	axon-spring-boot-autoconfigure-4.11.1	None	None	True	None	None	3	
9	axon-spring-boot-autoconfigure-4.11.1	None	None	True	None	None	4	

1.2.3 Longest path for each artifact

In this section we'll focus only on pairs of nodes that both belong to the same artifact, filtering out every line that has `isDifferentTargetProject==False` . The first ten rows are shown in a table followed by charts that show the distribution of longest path distances across different artifacts in stacked bar charts (absolute and normalized).

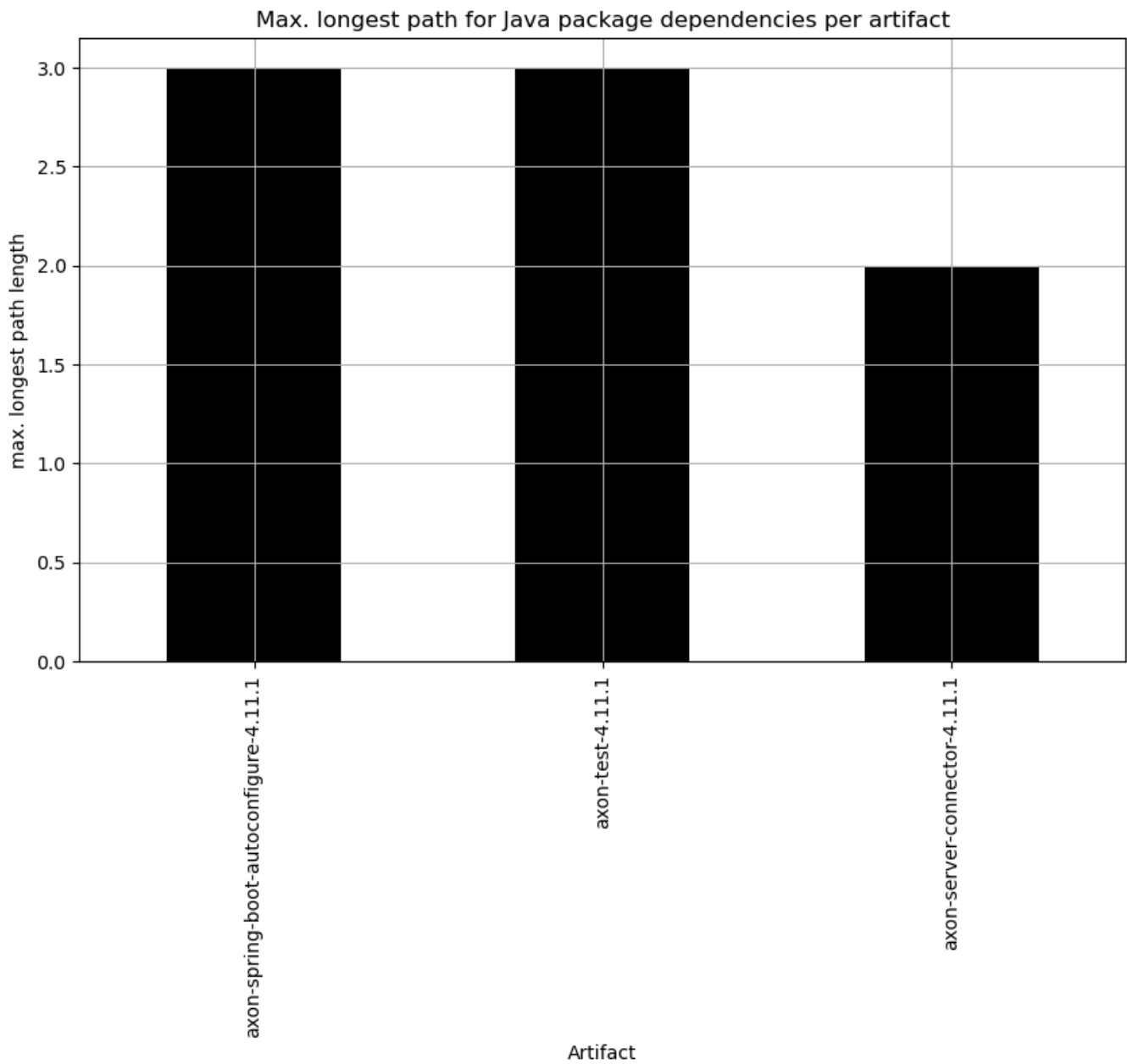
Note: It is possible that a (longest) path could have nodes in between that belong to different artifacts. Therefore, the data of each artifact isn't perfectly isolated. However, it shows how the dependencies interact across artifacts "in real life" while still providing a decent isolation of each artifact.

	sourceProject	sourceScan	sourceRootProject	isDifferentTargetProject	isDifferentTargetScan	isDifferentTargetRootProject	distance	dista
1	axon-server-connector-4.11.1	None	None	False	None	None	1	
2	axon-server-connector-4.11.1	None	None	False	None	None	2	
3	axon-spring-boot-autoconfigure-4.11.1	None	None	False	None	None	1	
4	axon-spring-boot-autoconfigure-4.11.1	None	None	False	None	None	2	
5	axon-spring-boot-autoconfigure-4.11.1	None	None	False	None	None	3	
11	axon-test-4.11.1	None	None	False	None	None	1	
12	axon-test-4.11.1	None	None	False	None	None	2	
13	axon-test-4.11.1	None	None	False	None	None	3	

Longest path for each artifact - Max. longest path for each artifact

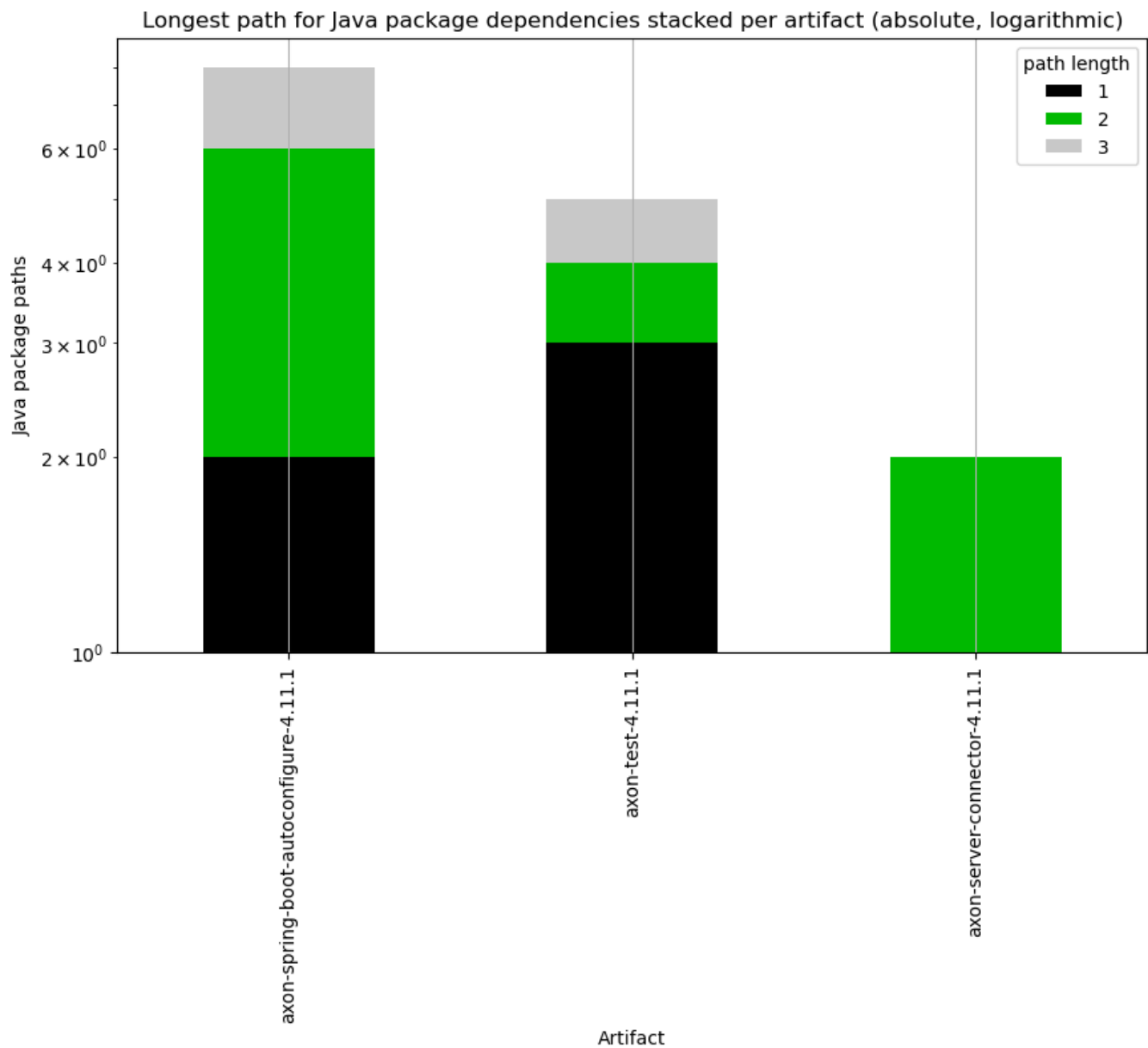
Shows the top 20 artifacts with their max. longest path.

```
sourceProject
axon-spring-boot-autoconfigure-4.11.1    3
axon-test-4.11.1                        3
axon-server-connector-4.11.1             2
Name: distance, dtype: int64
```



Longest path for each artifact - Bar chart (absolute)

<Figure size 640x480 with 0 Axes>

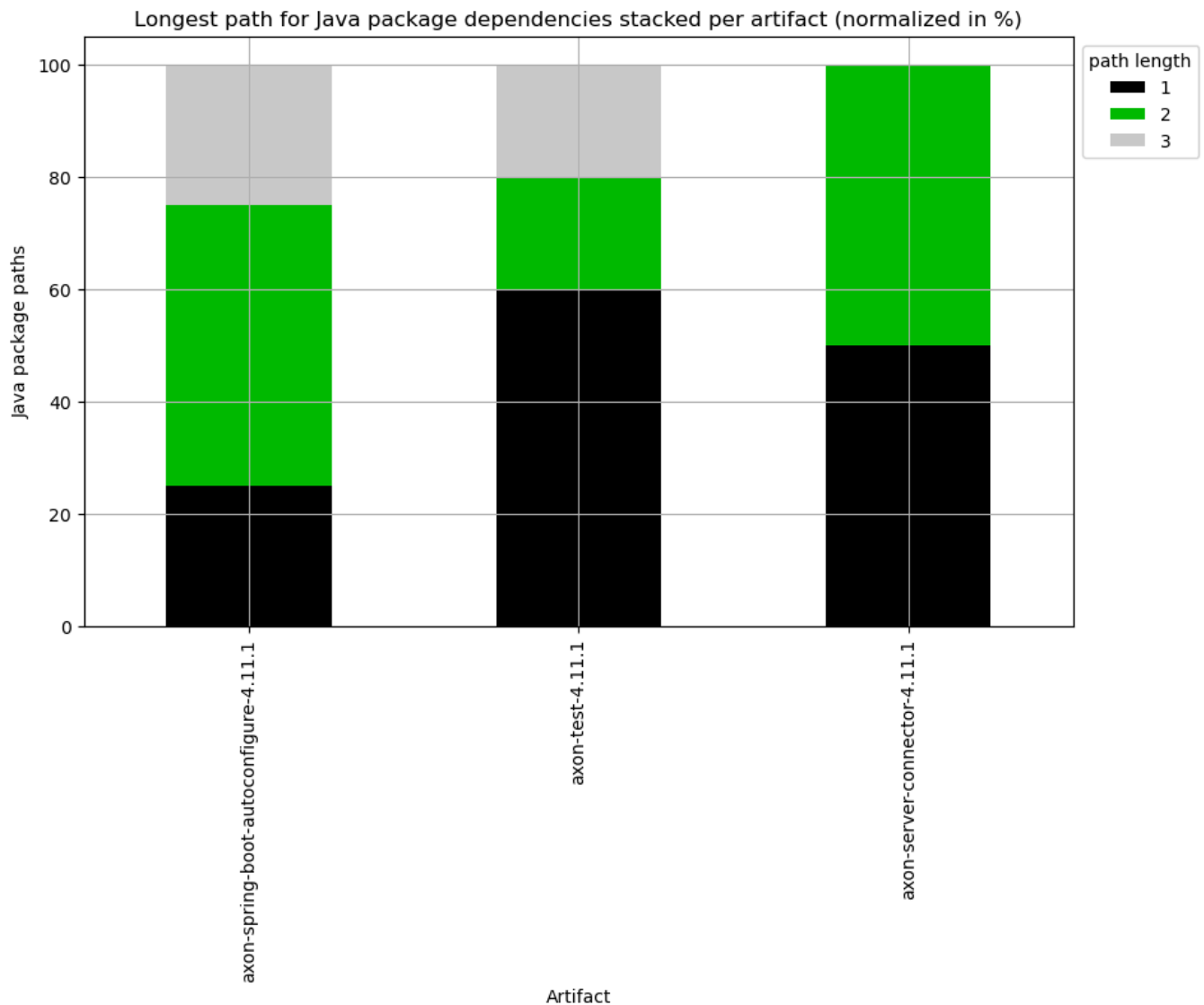


Longest path for each artifact - Bar chart (normalized)

Shows the top 50 artifacts with the highest number of dependency paths stacked by their length.

	distance	1	2	3
sourceProject				
axon-spring-boot-autoconfigure-4.11.1	25.0	50.0	25.0	
axon-test-4.11.1	60.0	20.0	20.0	
axon-server-connector-4.11.1	50.0	50.0	0.0	

<Figure size 640x480 with 0 Axes>



2. Java Artifacts

2.1 All pairs shortest path

Use [All Pairs Shortest Path](#) algorithm to get the shortest path distance between all pairs of dependent Java artifacts. It shows how many artifacts have a direct dependency (distance 1), how many are reachable with one dependency in between (distance 2), and so on...

2.1.1 Create a projection of all Java artifacts dependencies

Creates a in-memory projection of "Java:Artifact" nodes and their "DEPENDS_ON" relationships as a preparation to run the Graph algorithms. The weight property is not used for now (September 2024) but may be needed for other algorithms/variants some time.

Projected Graph statistics for Java Artifact dependencies

nodeCount	relationshipCount	density	sizeInBytes	degreeDistribution.min	degreeDistribution.mean	degreeDistribution.max	degreeDistri
0	9	25	0.347222	2726147	0	2.777778	7

2.1.2 All pairs shortest path in total

First, we'll have a look at the overall/total result of the all pairs shortest path algorithm for all dependencies.

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownRelationshipTypeWarning} {category: UNRECOGNIZED} {title: The provided relationship type is not in the database.} {description: One of the relationship types in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing relationship type is: CONTAINS_PROJECT)} {position: line: 31, column: 39, offset: 1603} for query: "// Path Finding - All pairs shortest path algorithm - Stream - Per project\n\n  CALL gds.allShortestPaths.stream($dependencies_projection + '-cleaned')\n  YIELD sourceNodeId, targetNodeId, distance\n  // Filter out all pairs that have no connection (infinite distance)\n  WHERE gds.util.isFinite(distance) = true\n  WITH toInteger(distance) AS distance\n  ,sourceNodeId\n  ,targetNodeId\n  WHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n  // Group by distance to get the overall distribution\n  WITH distance\n  ,count(*)\n  AS distanceTotalPairCount\n  ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n  ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n  ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n  // Unwind group to get every corresponding distance, source and target again\n  UNWIND sourcesAndTargets AS sourceAndTarget\n  WITH *\n  ,sourceAndTarget.sourceNodeId\n  AS sourceNodeId\n  ,sourceAndTarget.targetNodeId AS targetNodeId\n  // Resolve node ids to actual nodes\n  WITH *\n  ,gds.util.asNode(sourceNodeId) AS source\n  ,gds.util.asNode(targetNodeId) AS target\n  // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n  OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n  OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n  // Optionally get the name of the scan that contained that project\n  OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n  OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n  // Group by project name, if the target project is the same and the distance. Return those as result.\n  RETURN sourceProject.name\n  AS sourceProject\n  ,sourceScan.name\n  AS sourceScan\n  ,source.rootProjectName\n  AS sourceRootProject\n  ,(targetProject <> sourceProject) AS isDifferentTargetProject\n  ,(targetScan <> sourceScan) AS isDifferentTargetScan\n  ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n  ,distance\n  ,distanceTotalPairCount\n  ,distanceTotalSourceCount\n  ,distanceTotalTargetCount\n  ,count(*)\n  AS pairCount\n  ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n  ,count(DISTINCT targetNodeId) AS targetNodeCount\n  ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n  ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n  ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n  // Sort by source project name, if the target project is the same and the distance, all ascending\n  ORDER BY sourceProject, isDifferentTargetProject, distance"
```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: rootProjectName)} {position: line: 38, column: 42, offset: 2086} for query: "// Path Finding - All pairs shortest path algorithm - Stream - Per project\n\n CALL gds.allShortestPaths.stream(\$dependencies_projection + '-cleaned')\n YIELD sourceNodeId, targetNodeId, distance\n // Filter out all pairs that have no connection (infinite distance)\n WHERE gds.util.isFinite(distance) = true\n WITH toInteger(distance) AS distance\n ,sourceNodeId\n ,targetNodeId\n WHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n // Group by distance to get the overall distribution\n WITH distance\n ,count(*) AS distanceTotalPairCount\n ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n // Unwind group to get every corresponding distance, source and target again\n UNWIND sourcesAndTargets AS sourceAndTarget\n WITH *\n ,sourceAndTarget.sourceNodeId AS sourceNodeId\n ,sourceAndTarget.targetNodeId AS targetNodeId\n // Resolve node ids to actual nodes\n WITH *\n ,gds.util.asNode(sourceNodeId) AS source\n ,gds.util.asNode(targetNodeId) AS target\n // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n // Optionally get the name of the scan that contained that project\n OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n // Group by project name, if the target project is the same and the distance. Return those as result.\n RETURN sourceProject.name AS sourceProject\n ,sourceScan.name AS sourceScan\n ,source.rootProjectName AS sourceRootProject\n ,(targetProject <> sourceProject) AS isDifferentTargetProject\n ,(targetScan <> sourceScan) AS isDifferentTargetScan\n ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n ,distance\n ,distanceTotalPairCount\n ,distanceTotalSourceCount\n ,distanceTotalTargetCount\n ,count(*) AS pairCount\n ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n ,count(DISTINCT targetNodeId) AS targetNodeCount\n ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n // Sort by source project name, if the target project is the same and the distance, all ascending\n ORDER BY sourceProject, isDifferentTargetProject, distance"

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: rootProjectName)} {position: line: 38, column: 16, offset: 2060} for query: "// Path Finding - All pairs shortest path algorithm - Stream - Per project\n\n CALL gds.allShortestPaths.stream(\$dependencies_projection + '-cleaned')\n YIELD sourceNodeId, targetNodeId, distance\n // Filter out all pairs that have no connection (infinite distance)\n WHERE gds.util.isFinite(distance) = true\n WITH toInteger(distance) AS distance\n ,sourceNodeId\n ,targetNodeId\n WHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n // Group by distance to get the overall distribution\n WITH distance\n ,count(*) AS distanceTotalPairCount\n ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n // Unwind group to get every corresponding distance, source and target again\n UNWIND sourcesAndTargets AS sourceAndTarget\n WITH *\n ,sourceAndTarget.sourceNodeId AS sourceNodeId\n ,sourceAndTarget.targetNodeId AS targetNodeId\n // Resolve node ids to actual nodes\n WITH *\n ,gds.util.asNode(sourceNodeId) AS source\n ,gds.util.asNode(targetNodeId) AS target\n // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n // Optionally get the name of the scan that contained that project\n OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n // Group by project name, if the target project is the same and the distance. Return those as result.\n RETURN sourceProject.name AS sourceProject\n ,sourceScan.name AS sourceScan\n ,source.rootProjectName AS sourceRootProject\n ,(targetProject <> sourceProject) AS isDifferentTargetProject\n ,(targetScan <> sourceScan) AS isDifferentTargetScan\n ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n ,distance\n ,distanceTotalPairCount\n ,distanceTotalSourceCount\n ,distanceTotalTargetCount\n ,count(*) AS pairCount\n ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n ,count(DISTINCT targetNodeId) AS targetNodeCount\n ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n // Sort by source project name, if the target project is the same and the distance, all ascending\n ORDER BY sourceProject, isDifferentTargetProject, distance"

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: rootProjectName)} {position: line: 35, column: 15, offset: 1865} for query: "// Path Finding - All pairs shortest path algorithm - Stream - Per project\n\n CALL gds.allShortestPaths.stream(\$dependencies_projection + '-cleaned')\n YIELD sourceNodeId, targetNodeId, distance\n // Filter out all pairs that have no connection (infinite distance)\n WHERE gds.util.isFinite(distance) = true\n WITH toInteger(distance) AS distance\n ,sourceNodeId\n ,targetNodeId\n WHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n // Group by distance to get the overall distribution\n WITH distance\n ,count(*) AS distanceTotalPairCount\n ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n // Unwind group to get every corresponding distance, source and target again\n UNWIND sourcesAndTargets AS sourceAndTarget\n WITH *\n ,sourceAndTarget.sourceNodeId AS sourceNodeId\n ,sourceAndTarget.targetNodeId AS targetNodeId\n // Resolve node ids to actual nodes\n WITH *\n ,gds.util.asNode(sourceNodeId) AS source\n ,gds.util.asNode(targetNodeId) AS target\n // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n // Optionally get the name of the scan that contained that project\n OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n // Group by project name, if the target project is the same and the distance. Return those as result.\n RETURN sourceProject.name AS sourceProject\n ,sourceScan.name AS sourceScan\n ,source.rootProjectName AS sourceRootProject\n ,(targetProject <> sourceProject) AS isDifferentTargetProject\n ,(targetScan <> sourceScan) AS isDifferentTargetScan\n ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n ,distance\n ,distanceTotalPairCount\n ,distanceTotalSourceCount\n ,distanceTotalTargetCount\n ,count(*) AS pairCount\n ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n ,count(DISTINCT targetNodeId) AS targetNodeCount\n ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n // Sort by source project name, if the target project is the same and the distance, all ascending\n ORDER BY sourceProject, isDifferentTargetProject, distance"

```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownRelationshipTypeWarning} {category: UNRECOGNIZED} {title: The provided relationship type is not in the database.} {description: One of the relationship types in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing relationship type is: CONTAINS_PROJECT)} {position: line: 30, column: 39, offset: 1530} for query: "// Path Finding - All pairs shortest path algorithm - Stream - Per project\n\n  CALL gds.allShortestPaths.stream($dependencies_projection + '-cleaned')\n  YIELD sourceNodeId, targetNodeId, distance\n  // Filter out all pairs that have no connection (infinite distance)\n  WHERE gds.util.isFinite(distance) = true\n  WITH toInteger(distance) AS distance\n  ,sourceNodeId\n  ,targetNodeId\n  WHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n  // Group by distance to get the overall distribution\n  WITH distance\n  ,count(*)\n  AS distanceTotalPairCount\n  ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n  ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n  ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n  // Unwind group to get every corresponding distance, source and target again\n  UNWIND sourcesAndTargets AS sourceAndTarget\n  WITH *\n  ,sourceAndTarget.sourceNodeId\n  AS sourceNodeId\n  ,sourceAndTarget.targetNodeId AS targetNodeId\n  // Resolve node ids to actual nodes\n  WITH *\n  ,gds.util.asNode(sourceNodeId) AS source\n  ,gds.util.asNode(targetNodeId) AS target\n  // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n  OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n  OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n  // Optionally get the name of the scan that contained that project\n  OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n  OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n  // Group by project name, if the target project is the same and the distance. Return those as result.\n  RETURN sourceProject.name\n  AS sourceProject\n  ,sourceScan.name\n  AS sourceScan\n  ,source.rootProjectName\n  AS sourceRootProject\n  ,(targetProject <> sourceProject) AS isDifferentTargetProject\n  ,(targetScan <> sourceScan) AS isDifferentTargetScan\n  ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n  ,distance\n  ,distanceTotalPairCount\n  ,distanceTotalSourceCount\n  ,distanceTotalTargetCount\n  ,count(*) AS pairCount\n  ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n  ,count(DISTINCT targetNodeId) AS targetNodeCount\n  ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n  ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n  ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n  // Sort by source project name, if the target project is the same and the distance, all ascending\n  ORDER BY sourceProject, isDifferentTargetProject, distance"

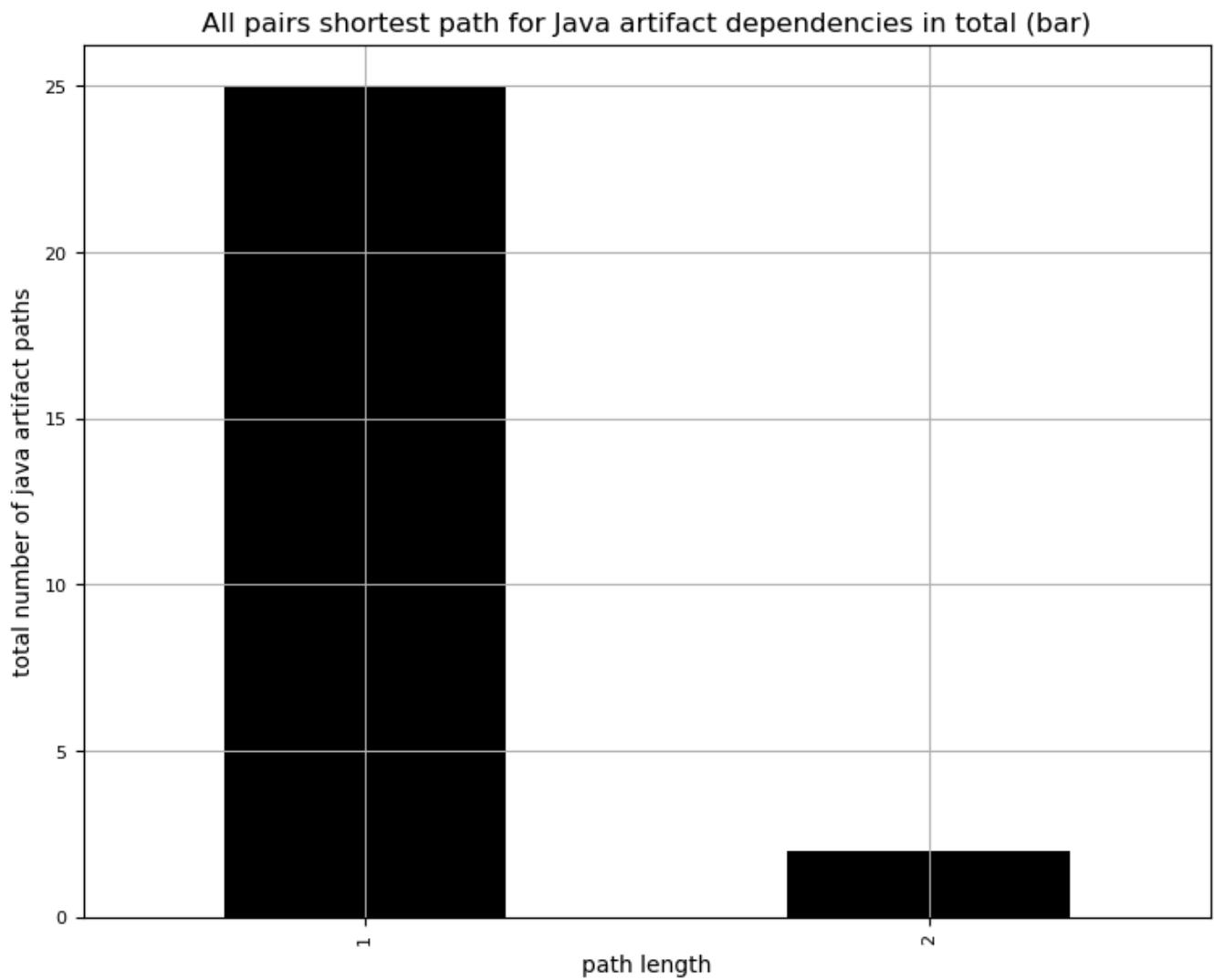
```

index	distance	distanceTotalPairCount	distanceTotalSourceCount	distanceTotalTargetCount
0	0	1	25	8
1	1	2	2	1

All pairs shortest path in total - Longest shortest path (Diameter)

The diameter (longest shortest path) of the projected artifact dependencies Graph is:
2

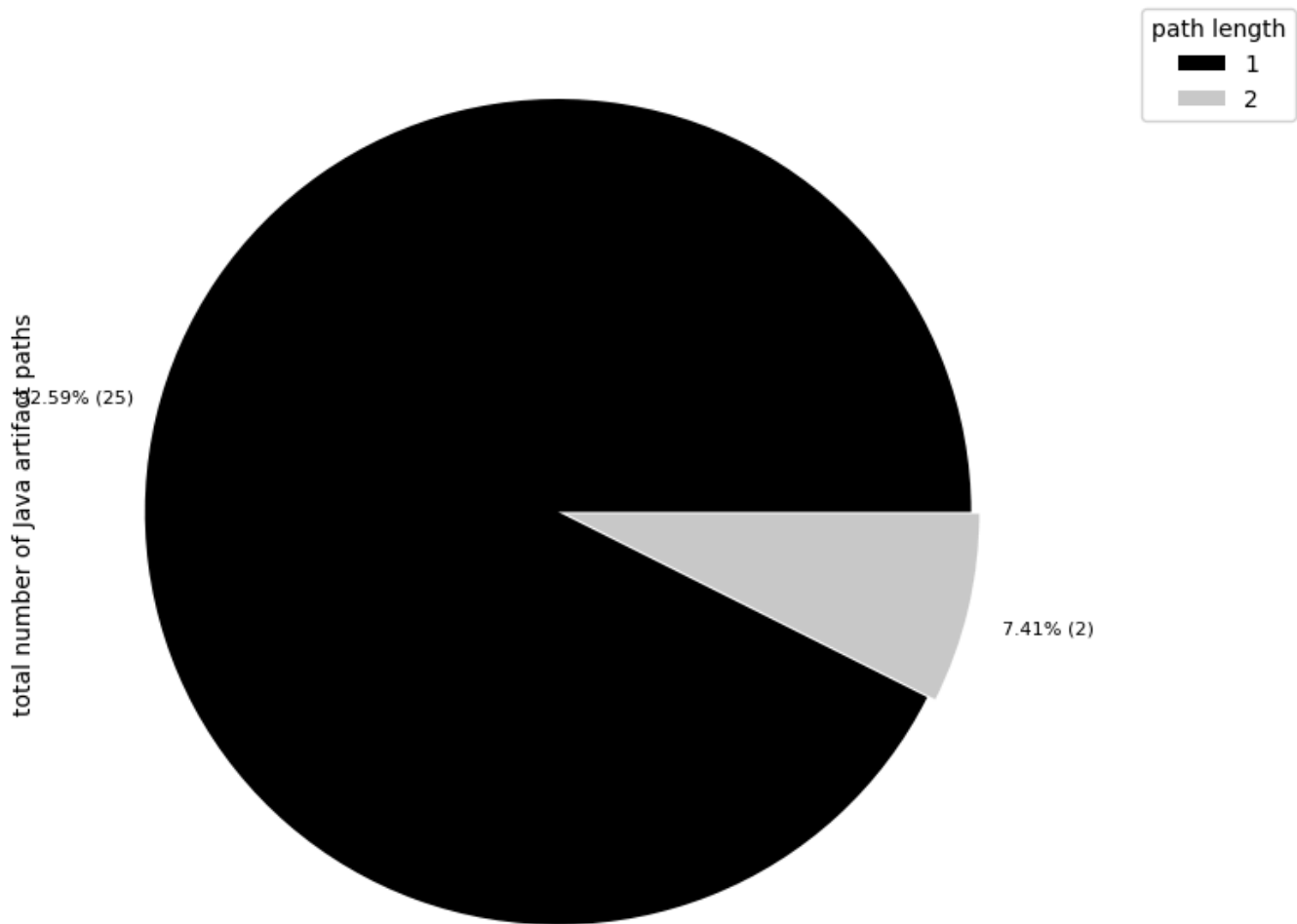
All pairs shortest path in total - Path count per length - Bar chart



All pairs shortest path in total - Path count per length - Bar chart

<Figure size 640x480 with 0 Axes>

All pairs shortest path for Java artifact dependencies in total (pie)



2.2 Longest path

Use [Longest Path](#) algorithm to get the longest paths between Java artifacts. This is especially interesting because an artifact can only be built when the artifacts it depends on are built (and published). The longest path shows in this case the total/max. number of build levels that are needed in case everything needs to be rebuild ("worst case"). Its also an interesting metric for complexity. It is typically higher than the longest shortest path (diameter) and helps together with it to get a good overview of the complexity.

Note: This algorithm requires a Directed Acyclic Graph (DAG) and might lead to inaccurate results when the Graph contains cycles.

2.2.1 Longest path in total

First, we'll have a look at the overall/total result of the longest path algorithm for all dependencies.

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: rootProjectName)} {position: line: 36, column: 42, offset: 2007} for query: "// Longest paths distribution\n\n    CALL gds.dag.longestPath.stream($dependencies_projection + '-cleaned')\n    YIELD index, sourceNode, targetNode, totalCost//,\n    nodeIds, costs, path\n    WITH toInteger(totalCost) AS distance\n        ,sourceNode\n    AS sourceNodeId\n        ,targetNode\n    AS targetNodeId\n    WHERE sourceNodeId\n<> targetNodeId // Filter out cyclic dependencies\n// Group by distance to get the overall distribution\n    WITH distance\n        ,count(*)\n    AS distanceTotalPairCount\n        ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n        ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n        ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n// Unwind group to get every corresponding distance, source and target again\nUNWIND sourcesAndTargets AS sourceAndTarget\n    WITH *\n        ,sourceAndTarget.sourceNodeId AS sourceNodeId\n        ,sourceAndTarget.targetNodeId AS targetNodeId\n// Resolve node ids to actual nodes\n    WITH *\n        ,gds.util.asNode(sourceNodeId) AS source\n        ,gds.util.asNode(targetNodeId) AS target\n// Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\nOPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\nOPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n// Optionally get the name of the scan that contained that project\nOPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\nOPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n// Group by project name, if the target project is the same and the distance. Return those as result.\nRETURN sourceProject.name\n        AS sourceProject\n        ,sourceScan.name\n        AS sourceScan\n        ,source.rootProjectName\n        AS sourceRootProject\n        ,(targetProject <> sourceProject) AS isDifferentTargetProject\n        ,(targetScan <> sourceScan) AS isDifferentTargetScan\n        ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n        ,distance\n        ,distanceTotalPairCount\n        ,distanceTotalSourceCount\n        ,distanceTotalTargetCount\n        ,count(*)\n    AS pairCount\n        ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n        ,count(DISTINCT targetNodeId) AS targetNodeCount\n        ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n        ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n        ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n// Sort by source project name, if the target project is the same and the distance, all ascending\nORDER BY sourceProject, isDifferentTargetProject, distance\n\n\n\n\n\n//RETURN toInteger(totalCost) AS totalCost\n//\n        ,count(*)\n    AS nodeCount\n//ORDER BY totalCost"
```

```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: rootProjectName)} {position: line: 36, column: 16, offset: 1981} for query: "// Longest paths distribution\n\n  CALL gds.dag.longestPath.stream($dependencies_projection + '-cleaned')\n  YIELD index, sourceNode, targetNode, totalCost//,\n  nodeIds, costs, path\n  WITH toInteger(totalCost) AS distance\n  ,sourceNode\n  AS sourceNodeId\n  ,targetNode\n  AS targetNodeId\n  WHERE sourceNodeId\n  <> targetNodeId // Filter out cyclic dependencies\n  // Group by distance to get the overall distribution\n  WITH distance\n  ,count(*)\n  AS distanceTotalPairCount\n  ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n  ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n  ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n  // Unwind group to get every corresponding distance, source and target again\n  UNWIND sourcesAndTargets AS sourceAndTarget\n  WITH *\n  ,sourceAndTarget.sourceNodeId AS sourceNodeId\n  ,sourceAndTarget.targetNodeId AS targetNodeId\n  // Resolve node ids to actual nodes\n  WITH *\n  ,gds.util.asNode(sourceNodeId) AS source\n  ,gds.util.asNode(targetNodeId) AS target\n  // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n  OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n  OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n  // Optionally get the name of the scan that contained that project\n  OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n  OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n  // Group by project name, if the target project is the same and the distance. Return those as result.\n  RETURN sourceProject.name\n  AS sourceProject\n  ,sourceScan.name\n  AS sourceScan\n  ,source.rootProjectName\n  AS sourceRootProject\n  ,(targetProject <> sourceProject) AS isDifferentTargetProject\n  ,(targetScan <> sourceScan)\n  AS isDifferentTargetScan\n  ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n  ,distance\n  ,distanceTotalPairCount\n  ,distanceTotalSourceCount\n  ,distanceTotalTargetCount\n  ,count(*)\n  AS pairCount\n  ,count(DISTINCT sourceNodeId)\n  AS sourceNodeCount\n  ,count(DISTINCT targetNodeId)\n  AS targetNodeCount\n  ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4]\n  AS examples\n  ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4]\n  AS exampleProjects\n  ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4]\n  AS exampleScans\n  // Sort by source project name, if the target project is the same and the distance, all ascending\n  ORDER BY sourceProject, isDifferentTargetProject, distance\n  \n\n\n\n\n\n\n\n\n  //RETURN toInteger(totalCost) AS totalCost\n  //\n  ,count(*)\n  AS nodeCount\n  //ORDER BY totalCost"

```



```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownRelationshipTypeWarning} {category: UNRECOGNIZED} {title: The provided relationship type is not in the database.} {description: One of the relationship types in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing relationship type is: CONTAINS_PROJECT)} {position: line: 29, column: 39, offset: 1524} for query: "// Longest paths distribution\n\n    CALL gds.dag.longestPath.stream($dependencies_projection + '-cleaned')\n    YIELD index, sourceNode, targetNode, totalCost//, nodeIds, costs, path\n    WITH toInteger(totalCost) AS distance\n    ,sourceNode AS sourceNodeId\n    ,targetNode AS targetNodeId\nWHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n// Group by distance to get the overall distribution\n    WITH distance\n    ,count(*)\nAS distanceTotalPairCount\n    ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n    ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n    ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n// Unwind group to get every corresponding distance, source and target again\nUNWIND sourcesAndTargets AS sourceAndTarget\n    WITH *\n    ,sourceAndTarget.sourceNodeId AS sourceNodeId\n    ,sourceAndTarget.targetNodeId AS targetNodeId\n// Resolve node ids to actual nodes\n    WITH *\n    ,gds.util.asNode(sourceNodeId) AS source\n    ,gds.util.asNode(targetNodeId) AS target\n// Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\nOPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\nOPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n// Optionally get the name of the scan that contained that project\nOPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\nOPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n// Group by project name, if the target project is the same and the distance. Return those as result.\nRETURN sourceProject.name AS sourceProject\n    ,sourceScan.name AS sourceScan\n    ,source.rootProjectName AS sourceRootProject\n    ,(targetProject <> sourceProject) AS isDifferentTargetProject\n    ,(targetScan <> sourceScan) AS isDifferentTargetScan\n    ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n    ,distance\n    ,distanceTotalPairCount\n    ,distanceTotalSourceCount\n    ,distanceTotalTargetCount\n    ,count(*) AS pairCount\n    ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n    ,count(DISTINCT targetNodeId) AS targetNodeCount\n    ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n    ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n    ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n// Sort by source project name, if the target project is the same and the distance, all ascending\nORDER BY sourceProject, isDifferentTargetProject, distance\n\n\n\n\n\n//RETURN toInteger(totalCost) AS totalCost\n//    ,count(*) AS nodeCount\n//ORDER BY totalCost"

```

```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: rootProjectName)} {position: line: 33, column: 15, offset: 1786} for query: "// Longest paths distribution\n\n  CALL gds.dag.longestPath.stream($dependencies_projection + '-cleaned')\n  YIELD index, sourceNode, targetNode, totalCost//,\n  nodeIds, costs, path\n  WITH toInteger(totalCost) AS distance\n  ,sourceNode\n  AS sourceNodeId\n  ,targetNode\n  AS targetNodeId\n  WHERE sourceNodeId\n  <> targetNodeId // Filter out cyclic dependencies\n  // Group by distance to get the overall distribution\n  WITH distance\n  ,count(*)\n  AS distanceTotalPairCount\n  ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n  ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n  ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n  // Unwind group to get every corresponding distance, source and target again\n  UNWIND sourcesAndTargets AS sourceAndTarget\n  WITH *\n  ,sourceAndTarget.sourceNodeId AS sourceNodeId\n  ,sourceAndTarget.targetNodeId AS targetNodeId\n  // Resolve node ids to actual nodes\n  WITH *\n  ,gds.util.asNode(sourceNodeId) AS source\n  ,gds.util.asNode(targetNodeId) AS target\n  // Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\n  OPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\n  OPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n  // Optionally get the name of the scan that contained that project\n  OPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\n  OPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n  // Group by project name, if the target project is the same and the distance. Return those as result.\n  RETURN sourceProject.name\n  AS sourceProject\n  ,sourceScan.name\n  AS sourceScan\n  ,source.rootProjectName\n  AS sourceRootProject\n  ,(targetProject <> sourceProject) AS isDifferentTargetProject\n  ,(targetScan <> sourceScan)\n  AS isDifferentTargetScan\n  ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n  ,distance\n  ,distanceTotalPairCount\n  ,distanceTotalSourceCount\n  ,distanceTotalTargetCount\n  ,count(*)\n  AS pairCount\n  ,count(DISTINCT sourceNodeId)\n  AS sourceNodeCount\n  ,count(DISTINCT targetNodeId)\n  AS targetNodeCount\n  ,collect(DISTINCT source.fileName\n  + ' -> ' + target.fileName)[0..4]\n  AS examples\n  ,collect(DISTINCT sourceProject.name\n  + ' -> ' + targetProject.name)[0..4]\n  AS exampleProjects\n  ,collect(DISTINCT sourceScan.name\n  + ' -> ' + targetScan.name)[0..4]\n  AS exampleScans\n  // Sort by source project name, if the target project is the same and the distance, all ascending\n  ORDER BY sourceProject, isDifferentTargetProject, distance\n  \n\n\n\n\n\n\n\n\n  //RETURN toInteger(totalCost) AS totalCost\n  //\n  ,count(*)\n  AS nodeCount\n  //ORDER BY totalCost"

```



```

Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownRelationshipTypeWarning} {category: UNRECOGNIZED} {title: The provided relationship type is not in the database.} {description: One of the relationship types in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing relationship type is: CONTAINS_PROJECT)} {position: line: 28, column: 39, offset: 1451} for query: "// Longest paths distribution\n\n    CALL gds.dag.longestPath.stream($dependencies_projection + '-cleaned')\n    YIELD index, sourceNode, targetNode, totalCost//, nodeIds, costs, path\n    WITH toInteger(totalCost) AS distance\n    ,sourceNode AS sourceNodeId\n    ,targetNode AS targetNodeId\nWHERE sourceNodeId <> targetNodeId // Filter out cyclic dependencies\n// Group by distance to get the overall distribution\n    WITH distance\n    ,count(*)\nAS distanceTotalPairCount\n    ,count(DISTINCT sourceNodeId) AS distanceTotalSourceCount\n    ,count(DISTINCT targetNodeId) AS distanceTotalTargetCount\n    ,collect({sourceNodeId: sourceNodeId, targetNodeId: targetNodeId}) AS sourcesAndTargets\n// Unwind group to get every corresponding distance, source and target again\nUNWIND sourcesAndTargets AS sourceAndTarget\n    WITH *\n    ,sourceAndTarget.sourceNodeId AS sourceNodeId\n    ,sourceAndTarget.targetNodeId AS targetNodeId\n// Resolve node ids to actual nodes\n    WITH *\n    ,gds.util.asNode(sourceNodeId) AS source\n    ,gds.util.asNode(targetNodeId) AS target\n// Optionally get the project (e.g. Java Artifact, Typescript Project) the source and target belong to\nOPTIONAL MATCH (sourceProject:Artifact|Project)-[:CONTAINS]->(source)\nOPTIONAL MATCH (targetProject:Artifact|Project)-[:CONTAINS]->(target)\n// Optionally get the name of the scan that contained that project\nOPTIONAL MATCH (sourceScan:TS:Scan)-[:CONTAINS_PROJECT]->(sourceProject)\nOPTIONAL MATCH (targetScan:TS:Scan)-[:CONTAINS_PROJECT]->(targetProject)\n// Group by project name, if the target project is the same and the distance. Return those as result.\nRETURN sourceProject.name AS sourceProject\n    ,sourceScan.name AS sourceScan\n    ,source.rootProjectName AS sourceRootProject\n    ,(targetProject <> sourceProject) AS isDifferentTargetProject\n    ,(targetScan <> sourceScan) AS isDifferentTargetScan\n    ,(target.rootProjectName <> source.rootProjectName) AS isDifferentTargetRootProject\n    ,distance\n    ,distanceTotalPairCount\n    ,distanceTotalSourceCount\n    ,distanceTotalTargetCount\n    ,count(*) AS pairCount\n    ,count(DISTINCT sourceNodeId) AS sourceNodeCount\n    ,count(DISTINCT targetNodeId) AS targetNodeCount\n    ,collect(DISTINCT source.fileName + ' -> ' + target.fileName)[0..4] AS examples\n    ,collect(DISTINCT sourceProject.name + ' -> ' + targetProject.name)[0..4] AS exampleProjects\n    ,collect(DISTINCT sourceScan.name + ' -> ' + targetScan.name)[0..4] AS exampleScans\n// Sort by source project name, if the target project is the same and the distance, all ascending\nORDER BY sourceProject, isDifferentTargetProject, distance\n\n\n\n\n\n//RETURN toInteger(totalCost) AS totalCost\n//    ,count(*) AS nodeCount\n//ORDER BY totalCost"

```

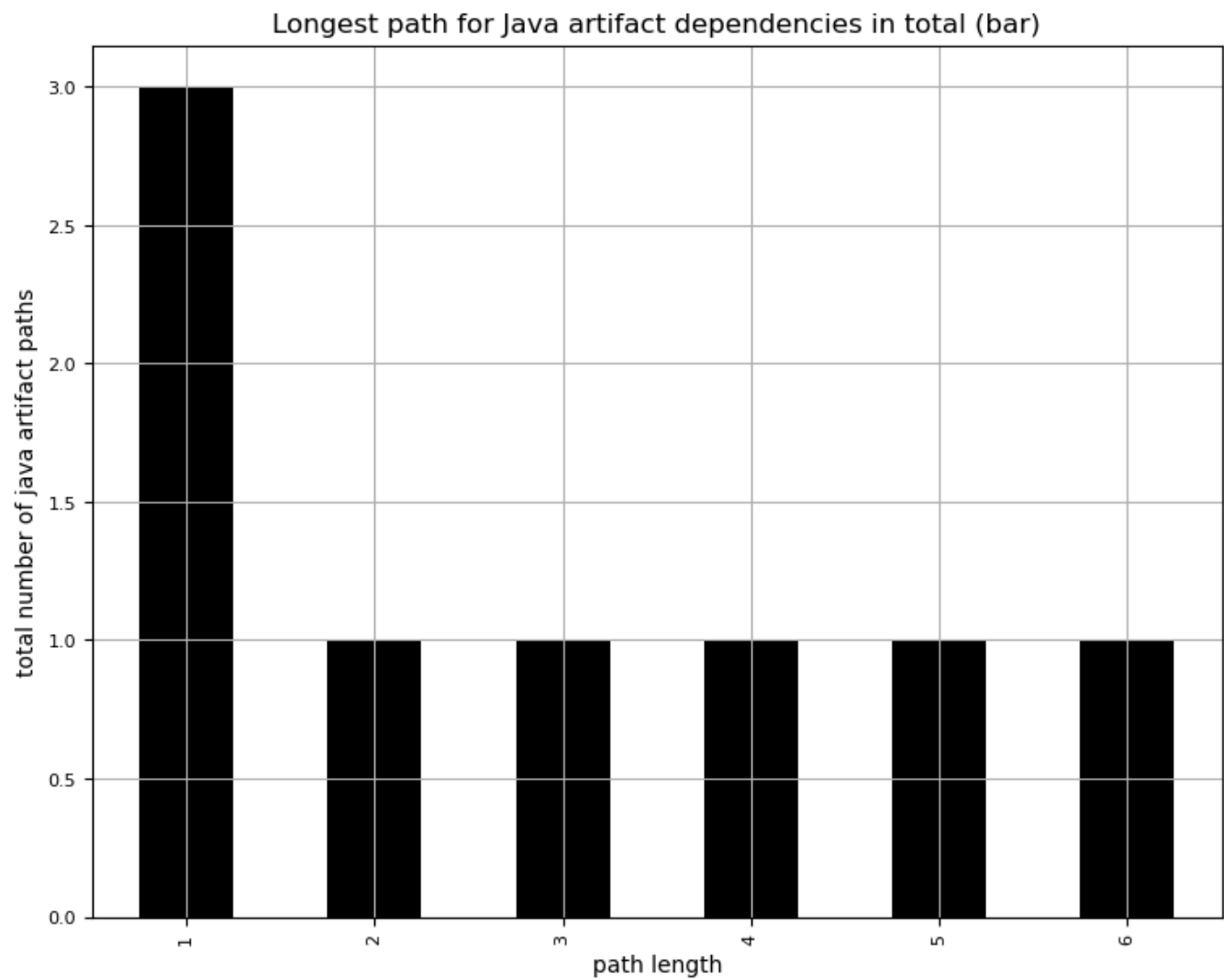
Longest path in total - Max longest path

The max. longest path of the projected artifact dependencies is: 6

Longest path in total - Path count per length - Table

	index	distance	distanceTotalPairCount	distanceTotalSourceCount	distanceTotalTargetCount
	0	0	1	3	1
	1	1	2	1	1
	2	2	3	1	1
	3	3	4	1	1
	4	4	5	1	1
	5	5	6	1	1

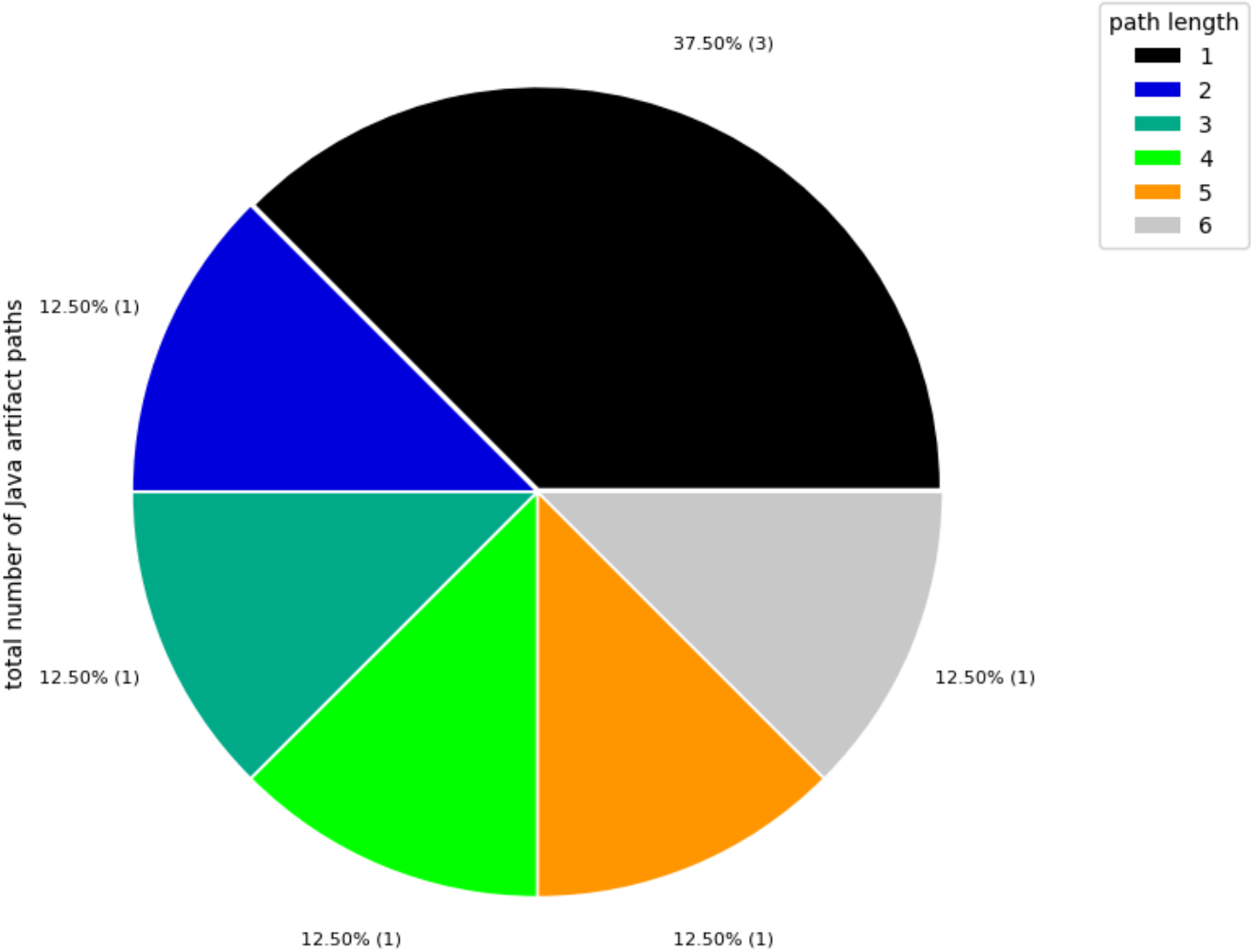
Longest path in total - Path count per length - Bar chart



Longest path in total - Path count per length - Pie chart

<Figure size 640x480 with 0 Axes>

Longest path for Java artifact dependencies in total (pie)



3. Summary

3.1 Java packages summary

	count	degree density	degree median	degree max	longest shortest path (diameter)	max. longest path
0	116	0.056972	5	58	5	5

3.2 Java artifacts summary

	count	degree density	degree median	degree max	longest shortest path (diameter)	max. longest path
0	9	0.347222	3	7	2	6